KERALA POLYTECHNIC · REVISION 2026

VLSI Testing

Complete student lesson for course code **5493D**.

Revision 2026 Semester 5 Program Elective course 4 modules

Course snapshot

Scheme: 4 (L:4 T:0 P:0)

Credits: 4

Contact: 60 periods per semester

Module hours listed: 58

Official Source Declaration

This lesson uses the supplied official syllabus PDF as the authoritative source for course information, revision, modules, topics, objectives, Course Outcomes, assessment information, official activities, practical requirements, and references.

Source handling: Official wording and numerical values are retained wherever supplied. Educational explanations, Malayalam support, examples, diagrams, and revision questions are added as study support and are not presented as additional official requirements.

Transparency note: The supplied PDF does not specify a separate CIE/ESE distribution. Any limitation in the supplied PDF is not silently corrected.

How to Study This Lesson

Recommended sequence

1. Begin with the official source declaration and course dashboard.

2. Study each module in the official order and keep the coverage table as a checklist.
3. Open every topic, understand the example or procedure, and write your own short answer.
4. Use the revision section, glossary, questions, and practice paper after completing the modules.

Study principle: Keep English technical terminology, symbols, units, and official assessment wording unchanged in examination answers. Use the Malayalam support blocks only as a bridge to understanding.

Handbook method: Do not treat a topic as completed after reading its heading. For every topic card, complete the learning objectives, follow the conceptual framework, write the worked answer method, and answer the self-check questions. This creates a study record that is useful for class learning and examination preparation.

Course Dashboard

Course code

5493D

Semester

5

Credits

4

Teaching scheme

4 (L:4 T:0 P:0)

Module-hour and outcome mapping

No.	Module	Official hours	Relevant CO / level
1	Introduction to VLSI Testing and Fault Models	11	CO1
2	Logic and Fault Simulation and Testability Measures	20	CO2

No.	Module	Official hours	Relevant CO / level
3	Test Generation for Combinational and Sequential Circuits	14	CO3
4	Design for Testability, BIST and Fault Diagnosis	13	CO4

Prerequisites

- Combinational Circuits — Digital Electronics — semester 3.
- Sequential Circuits and its types — Digital Electronics — semester 3.

How to study this lesson

1. Read the module introduction and learning goals.
2. Open every topic and reproduce its example, sequence, or procedure.
3. Use Malayalam support as a bridge; retain English technical terms for examinations.
4. Attempt the module questions without looking at the answers.

Objectives, Course Outcomes and CO–PO Information

Official course objectives

1. to introduce the VLSI testing
2. to introduce logic and fault simulation and testability measures
3. to study the test generation for combinational and sequential circuits
4. to study the design for testability.
5. to study the fault diagnosis

Course Outcomes

1. CO1: Understand VLSI Testing Process — 11 hours — Understanding.
2. CO2: Develop Logic Simulation and Fault Simulation — 20 hours — Understanding.
3. CO3: Develop Test for Combinational and Sequential Circuits — 14 hours — Understanding.
4. CO4: Understand the Design for Testability — 13 hours — Understanding.

CO-PO mapping as supplied

CO1: PO1 = 3 (Strongly mapped).
CO2: PO1 = 3 (Strongly mapped); PO2 = 2 (Moderately mapped).
CO3: PO1 = 3 (Strongly mapped); PO2 = 3 (Strongly mapped); PO3 = 3 (Strongly mapped).
CO4: PO1 = 3 (Strongly mapped); PO2 = 2 (Moderately mapped).
Legend supplied by the PDF: 3-Strongly mapped, 2-Moderately mapped, 1-Weakly mapped.

Complete Syllabus Coverage

Official syllabus point-by-point coverage

This audit layer maps every official source block to the corresponding lesson module. The source transcript is retained verbatim as extracted from the supplied syllabus; the study guidance below is educational support and is not presented as additional official syllabus content. No official point is intentionally omitted.

Source-to-lesson coverage map

Module	Lesson topic cards	Source basis	Official point units
Module 1	4	Official Contents block	3
Module 2	4	Official Contents block	3
Module 3	4	Official Contents block	4
Module 4	3	Official Contents block	5

Use this checklist to confirm that every official module topic is represented in the lesson.

Complete official topic coverage checklist

Module	Topic code	Topic	Hours
module-1	M1.01	Explain VLSI Testing Process	3
module-1	M1.02	Explain VLSI Testing Equipment	3
module-1	M1.03	Explain fault models	3
module-1	M1.04	Describe the relationship Among Fault Models.	2

Module	Topic code	Topic	Hours
module-2	M2.01	Explain Simulation for Design Verification and Test Evaluation	3
module-2	M2.02	Explain Modeling Circuits for Simulation	4
module-2	M2.03	Explain the algorithms for true value and fault simulation	6
module-2	M2.04	Find the faults in the given logic circuit	7
module-3	M3.01	Explain Combinational ATPG Algorithms	3
module-3	M3.02	Explain Sequential ATPG Algorithms	3
module-3	M3.03	Explain Simulation based ATPG	4
module-3	M3.04	Explain Genetic Algorithm Based ATPG	4
module-4	M4.01	Explain Testabilty basics and its analysis	4
module-4	M4.02	Explain BIST architecture	4
module-4	M4.03	Explain Fault Models for Diagnosis	5

Learning Roadmap

1. Introduction to VLSI Testing and Fault Models
CO1 · 11 hours

2. Logic and Fault Simulation and Testability Measures
CO2 · 20 hours

3. Test Generation for Combinational and Sequential Circuits
CO3 · 14 hours

4. Design for Testability, BIST and Fault Diagnosis
CO4 · 13 hours

The roadmap follows the order of the supplied syllabus.

OFFICIAL MODULE 1

Introduction to VLSI Testing and Fault Models

Testing is the evidence-based process of checking whether a manufactured VLSI circuit satisfies its intended behaviour and quality requirements. This module introduces the complete testing path, the equipment used to apply and observe tests, the economic consequences of defects, and the fault models that make complex physical defects manageable in digital analysis.

Hours: 11

CO1 · Bloom level: Understanding

Handbook study routine: Complete Module 1 in three passes. First read the official source coverage and topic headings. Next study every Handbook treatment, writing the key definition, relationship, procedure, formula, diagram, or comparison in your own words. Finally close the notes and answer the self-check questions and the module questions without looking at the answer.

Official source coverage for Module 1

Source basis: Official Contents block. Every point below is retained and linked to a study-oriented explanation. The main topic cards remain the primary lesson narrative.

View extracted official source transcript

s:
INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test
Equipment – Challenges in VLSI Testing - Test Economics and Product Quality –
Fault
Modeling – Relationship Among Fault Models.

Point-by-point study guide

Official syllabus point 1: INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test

Exact source point

Syllabus text: INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test

How to understand and study it

This point is an official syllabus requirement: “INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test”. Begin with the exact definition or meaning, then identify the purpose, scope, and terminology contained in the point. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Write the operating sequence in numbered steps, including the input, intermediate action, and output.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

Malayalam support: □ syllabus point □□□□□□□□□□□□ INTRODUCTION TO TESTING, Introduction – VLSI Testing Process □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

Exam focus: Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test using the official terminology retained above.
- Organise the important elements of INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test into a logical sequence, classification, structure, or calculation.
- Connect INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test with one engineering decision, operation, measurement, or safety requirement in the context of Introduction to VLSI Testing and Fault Models.
- Write an examination answer on INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test. It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test?
- How would you distinguish INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test from a closely related idea?
- Where would an engineer or technician encounter INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test, and what must be checked before using it?
- What common mistake would make an answer or operation involving INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: INTRODUCTION TO TESTING: Introduction – VLSI Testing Process and Test □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□.

Official syllabus point 2: Equipment – Challenges in VLSI Testing - Test Economics and Product Quality – Fault

Exact source point

Syllabus text: Equipment – Challenges in VLSI Testing - Test Economics and Product Quality – Fault

How to understand and study it

This point is an official syllabus requirement: “Equipment – Challenges in VLSI Testing - Test Economics and Product Quality – Fault”. Treat every named term as an examinable subpoint. Define it, explain its role or behaviour, distinguish it from related terms, and connect it to the module application. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. List the named terms separately and add one distinguishing feature or engineering use for each.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

Malayalam support: □ syllabus point □□□□□□□□□□□ Equipment – Challenges in VLSI Testing - Test Economics, Product Quality – Fault □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

Exam focus: Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

Equipment – Challenges in VLSI Testing - Test Economics and Product Quality – Fault is a decision and coordination topic. Learn the definitions, then connect the planning inputs, method, output, performance measure, and corrective action.

Learning objectives

- Define and scope Equipment – Challenges in VLSI Testing - Test Economics and Product Quality – Fault using the official terminology retained above.
- Organise the important elements of Equipment – Challenges in VLSI Testing - Test Economics and Product Quality – Fault into a logical sequence, classification, structure, or calculation.
- Connect Equipment – Challenges in VLSI Testing - Test Economics and Product Quality – Fault with one engineering decision, operation, measurement, or safety requirement in the context of Introduction to VLSI Testing and Fault Models.
- Write an examination answer on Equipment – Challenges in VLSI Testing - Test Economics and Product Quality – Fault with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading Equipment – Challenges in VLSI Testing - Test Economics and Product Quality – Fault. It is a study organisation method, not an additional official syllabus requirement.

- Define the management term or objective.
- Identify the resources, constraints, people, information, and time involved.
- Describe the procedure or decision criteria in a logical sequence.
- Measure the result and state how deviations are controlled or improved.

Engineering application lens

Engineering organisations apply Equipment – Challenges in VLSI Testing - Test Economics and Product Quality – Fault to deliver safe, economical, reliable, and timely work. A strong answer connects the method to productivity, quality, cost, schedule, customer requirement, compliance, or sustainability as appropriate.

Worked answer method

Given: the project, process, resource, or organisational situation.
Required: plan, comparison, decision, or performance explanation.
Method: state objective → inputs → method → output → measure → corrective action.
Answer: finish with the likely benefit, limitation, and condition for successful implementation.

Exam answer blueprint

For a short-answer question on Equipment – Challenges in VLSI Testing - Test Economics and Product Quality – Fault, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is Equipment – Challenges in VLSI Testing - Test Economics and Product Quality – Fault, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining Equipment – Challenges in VLSI Testing - Test Economics and Product Quality – Fault?
- How would you distinguish Equipment – Challenges in VLSI Testing - Test Economics and Product Quality – Fault from a closely related idea?
- Where would an engineer or technician encounter Equipment – Challenges in VLSI Testing - Test Economics and Product Quality – Fault, and what must be checked before using it?
- What common mistake would make an answer or operation involving Equipment – Challenges in VLSI Testing - Test Economics and Product Quality – Fault incorrect?

Common mistakes and safety emphasis

- Listing management terms without a sequence or decision criterion.
- Ignoring constraints, measurement, responsibility, or feedback.
- Confusing an objective with an activity or a result with an input.
- Giving a generic benefit without linking it to the engineering situation.

Malayalam support: Equipment – Challenges in VLSI Testing - Test Economics and Product Quality – Fault □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□.

Official syllabus point 3: Modeling – Relationship Among Fault Models.

Exact source point

Syllabus text: Modeling – Relationship Among Fault Models.

How to understand and study it

This point is an official syllabus requirement: “Modeling – Relationship Among Fault Models.”. Treat every named term as an examinable subpoint. Define it, explain its role or behaviour, distinguish it from related terms, and connect it to the module application. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. List the named terms separately and add one distinguishing feature or engineering use for each.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

Malayalam support: □ syllabus point □□□□□□□□□□□□ Modeling – Relationship Among Fault Models. □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

Exam focus: Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

Modeling – Relationship Among Fault Models. is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope Modeling – Relationship Among Fault Models. using the official terminology retained above.
- Organise the important elements of Modeling – Relationship Among Fault Models. into a logical sequence, classification, structure, or calculation.
- Connect Modeling – Relationship Among Fault Models. with one engineering decision, operation, measurement, or safety requirement in the context of Introduction to VLSI Testing and Fault Models.
- Write an examination answer on Modeling – Relationship Among Fault Models. with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading Modeling – Relationship Among Fault Models.. It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of Modeling – Relationship Among Fault Models. is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on Modeling – Relationship Among Fault Models., begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is Modeling – Relationship Among Fault Models., and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining Modeling – Relationship Among Fault Models.?
- How would you distinguish Modeling – Relationship Among Fault Models. from a closely related idea?
- Where would an engineer or technician encounter Modeling – Relationship Among Fault Models., and what must be checked before using it?
- What common mistake would make an answer or operation involving Modeling – Relationship Among Fault Models. incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

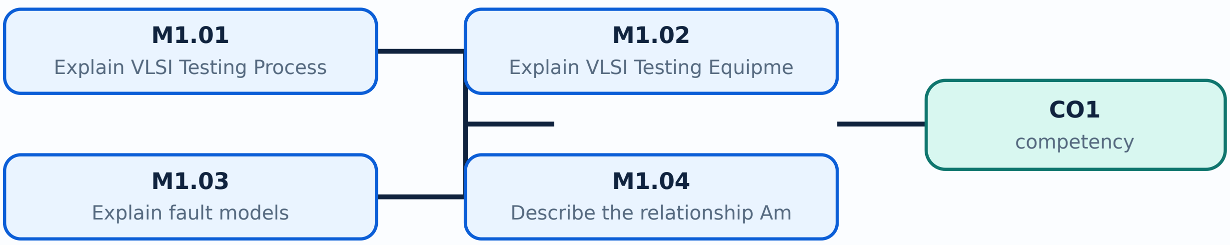
Malayalam support: Modeling – Relationship Among Fault Models. □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

Learning goals

- Describe the VLSI testing process from design intent through production test and quality feedback.
- Identify the role of automatic test equipment and the main challenges in testing dense VLSI circuits.
- Explain common fault models and the relationship between physical defects and logical abstractions.

Module study tip

Read every topic in sequence, reproduce its example or procedure, and write a short explanation before attempting the module questions.



Learning map for Module module-1: the listed topics build the module competency.

M1.01 — Explain VLSI Testing Process (3 hours)

Concept and explanation

The VLSI testing process converts a design specification into test requirements, derives test patterns, applies them to a device, captures the responses, compares the responses with expected values, and records pass/fail information. Testing may occur at wafer level, after packaging, during production screening, and during system-level diagnosis. A useful test flow separates defect prevention, design verification, manufacturing defect detection, quality grading, and failure analysis. Test coverage is the proportion of targeted faults detected by the applied test set; high coverage improves confidence but does not mean that every possible physical defect has been observed.

Malayalam support: VLSI testing □□□□□□ design □□□□□□□ □□□□□□□□□□□□□□□□□□□□□ □□□□□□ manufactured chip-□ defect □□□□□ □□□□□□ □□□□□□□□□□□□□□ process □□□. Test pattern apply □□□□□□ output expected value-□□□□□□ compare □□□□□□□□□□. High test coverage □□□□□□□□, □□□□□ □□□□□ physical defects-□□ □□□□□□□□□□□□□□□ □□□□□ □□□□□□□□□□□.

Study points

- Relate specification, test generation, pattern application, response capture, comparison, and diagnosis in the correct order.
- Distinguish design verification from manufacturing test.
- Define test coverage as a measure of detected targeted faults.

Engineering / workplace application: Semiconductor manufacturers use wafer probing, package testing, burn-in or screening, and failure analysis to maintain product quality across large production volumes.

Illustrative example: If a two-input gate is expected to implement AND, a test set must include input combinations that distinguish its correct truth table from likely stuck-at or bridging faults.

Common mistake: Treating functional simulation alone as a complete manufacturing test ignores defects introduced during fabrication and packaging.

Handbook treatment

M1.01 — Explain VLSI Testing Process (3 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope M1.01 — Explain VLSI Testing Process (3 hours) using the official terminology retained above.
- Organise the important elements of M1.01 — Explain VLSI Testing Process (3 hours) into a logical sequence, classification, structure, or calculation.
- Connect M1.01 — Explain VLSI Testing Process (3 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Introduction to VLSI Testing and Fault Models.
- Write an examination answer on M1.01 — Explain VLSI Testing Process (3 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading M1.01 — Explain VLSI Testing Process (3 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of M1.01 — Explain VLSI Testing Process (3 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on M1.01 — Explain VLSI Testing Process (3 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is M1.01 — Explain VLSI Testing Process (3 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining M1.01 — Explain VLSI Testing Process (3 hours)?
- How would you distinguish M1.01 — Explain VLSI Testing Process (3 hours) from a closely related idea?
- Where would an engineer or technician encounter M1.01 — Explain VLSI Testing Process (3 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving M1.01 — Explain VLSI Testing Process (3 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: M1.01 — Explain VLSI Testing Process (3 hours) [illegible] topic [illegible] [illegible] exact technical term [illegible]. [illegible] definition [illegible] principle, named parts/steps, application, limitation [illegible] [illegible] [illegible]. English terminology, symbols, units, diagram labels [illegible] [illegible]. Exam answer [illegible] concept-[illegible] [illegible]–[illegible] [illegible] [illegible].

M1.02 — Explain VLSI Testing Equipment (3 hours)

Concept and explanation

Automatic Test Equipment (ATE) supplies controlled input patterns, timing, power, and measurement functions to a device under test. A typical setup includes a tester controller, pattern memory, timing and signal-generation resources, power supplies, pin electronics, a load board or probe card, and response-analysis hardware. Equipment selection is influenced by pin count, operating frequency, voltage levels, analogue or mixed-signal content, memory depth, measurement accuracy, and cost. The equipment must control signal integrity and protect the device while collecting repeatable results.

Malayalam support: ATE □□□□□□ device under test-□□ patterns, timing, power □□□□□□ □□□□ response □□□□□□□□□ test system □□□. Tester, pattern memory, timing unit, power supply, pin electronics, probe card □□□□□□□□□□ load board □□□□□□ □□□□□□ □□□□□□□□□□.

Study points

- List the main functional blocks of an ATE setup.
- Explain why timing accuracy, pin count, voltage range, and measurement repeatability matter.
- Differentiate wafer probing from packaged-device testing at a high level.

Engineering / workplace application: ATE is used in wafer sort, final test, memory test, digital IC test, and production screening.

Illustrative example: A tester can apply a clocked pattern to a scan chain, capture the response, shift the captured bits out, and compare them with the expected signature.

Common mistake: Assuming that a low-cost logic analyser can replace production ATE overlooks timing, pin-parallelism, power, and automatic grading requirements.

Handbook treatment

M1.02 — Explain VLSI Testing Equipment (3 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope M1.02 — Explain VLSI Testing Equipment (3 hours) using the official terminology retained above.
- Organise the important elements of M1.02 — Explain VLSI Testing Equipment (3 hours) into a logical sequence, classification, structure, or calculation.
- Connect M1.02 — Explain VLSI Testing Equipment (3 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Introduction to VLSI Testing and Fault Models.
- Write an examination answer on M1.02 — Explain VLSI Testing Equipment (3 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading M1.02 — Explain VLSI Testing Equipment (3 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of M1.02 — Explain VLSI Testing Equipment (3 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on M1.02 — Explain VLSI Testing Equipment (3 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is M1.02 — Explain VLSI Testing Equipment (3 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining M1.02 — Explain VLSI Testing Equipment (3 hours)?
- How would you distinguish M1.02 — Explain VLSI Testing Equipment (3 hours) from a closely related idea?
- Where would an engineer or technician encounter M1.02 — Explain VLSI Testing Equipment (3 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving M1.02 — Explain VLSI Testing Equipment (3 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: M1.02 — Explain VLSI Testing Equipment (3 hours) [illegible] topic [illegible] [illegible] exact technical term [illegible]. [illegible] definition [illegible] principle, named parts/steps, application, limitation [illegible] [illegible] [illegible]. English terminology, symbols, units, diagram labels [illegible] [illegible]. Exam answer [illegible] concept-[illegible] [illegible]–[illegible] [illegible] [illegible].

M1.03 — Explain fault models (3 hours)

Concept and explanation

A fault model is a simplified logical representation of a physical defect. Common digital models include stuck-at-0 and stuck-at-1 faults, open faults, short or bridging faults, transistor-level defects, and delay faults. The stuck-at model assumes that a signal line remains permanently at one logic value even when the fault-free circuit would produce the other value. A model is useful when it preserves the defect behaviours that matter for test generation without requiring every transistor-level detail in every calculation.

Malayalam support: Physical defect-□□ analysis □□□□□□□ □□□□□□□□□□□□□ simplified logical representation □□□ fault model. Stuck-at-0 signal □□□□□□□□ 0 □□□ □□□□□□□□□□; stuck-at-1 □□□□□□□□ 1 □□□ □□□□□□□□□. Model □□□□□□□ defect-□□□□ □□□□□ details □□□□□□□□□□□□□□□□□.

Study points

- Define a fault model and explain why abstraction is needed.
- Compare stuck-at, bridging, open, and delay fault ideas.
- State that fault-free and faulty circuit responses must be distinguishable for detection.

Engineering / workplace application: Fault models guide ATPG tools, simulation experiments, coverage calculations, and diagnosis in digital VLSI production flows.

Illustrative example: To detect a line with a stuck-at-0 fault, a test must first justify the good circuit value of that line as 1 and then propagate the difference to an observable output.

Common mistake: A stuck-at fault is not the same as a logic value that happens to be 0 or 1 for one input pattern; it is a persistent abnormal constraint.

Handbook treatment

M1.03 — Explain fault models (3 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope M1.03 — Explain fault models (3 hours) using the official terminology retained above.
- Organise the important elements of M1.03 — Explain fault models (3 hours) into a logical sequence, classification, structure, or calculation.
- Connect M1.03 — Explain fault models (3 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Introduction to VLSI Testing and Fault Models.
- Write an examination answer on M1.03 — Explain fault models (3 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading M1.03 — Explain fault models (3 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of M1.03 — Explain fault models (3 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on M1.03 — Explain fault models (3 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is M1.03 — Explain fault models (3 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining M1.03 — Explain fault models (3 hours)?
- How would you distinguish M1.03 — Explain fault models (3 hours) from a closely related idea?
- Where would an engineer or technician encounter M1.03 — Explain fault models (3 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving M1.03 — Explain fault models (3 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: M1.03 — Explain fault models (3 hours) [illegible] topic [illegible] [illegible] exact technical term [illegible]. [illegible] definition [illegible] principle, named parts/steps, application, limitation [illegible] [illegible] [illegible]. English terminology, symbols, units, diagram labels [illegible] [illegible]. Exam answer [illegible] concept-[illegible] [illegible]–[illegible] [illegible] [illegible].

M1.04 — Describe the relationship Among Fault Models. (2 hours)

Concept and explanation

Fault models are related by abstraction and by the defects they can represent. A physical defect may be represented by one or more logical faults, and a single logical model may cover several physical causes. For example, a short or transistor defect can sometimes produce a stuck-at behaviour under a particular circuit condition, while a delay defect may escape a purely static stuck-at test. The relationship is therefore one of useful coverage and equivalence under assumptions, not a claim that all models are identical.

Malayalam support: □□□ physical defect □□ logical fault models □□□ represent □□□□□□□. □□□ □□□□□□□□□□□□□ physical defect stuck-at behaviour □□□□□□□□□□; delay fault static test □□□□□□ □□□□□□□□□□□□□□□ □□□□□. □□□□□□ models □□□□□□ complete equivalence □□□□, assumed behaviour □□□□□□□□□□□□□ relationship □□□.

Study points

- Explain why one physical defect can map to multiple logical behaviours.
- Distinguish fault equivalence under a test model from physical identity.
- Relate static fault models to delay and defect-specific models.

Engineering / workplace application: Engineers select a model set according to product risk, test cost, timing requirements, and the defect mechanisms expected from a fabrication technology.

Illustrative example: A transition that reaches the correct final logic value but arrives too late may pass a static test and fail a timing-sensitive test.

Common mistake: Claiming that 100% stuck-at coverage guarantees 100% coverage of delay, bridging, or analogue defects is technically incorrect.

Handbook treatment

M1.04 — Describe the relationship Among Fault Models. (2 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope M1.04 — Describe the relationship Among Fault Models. (2 hours) using the official terminology retained above.
- Organise the important elements of M1.04 — Describe the relationship Among Fault Models. (2 hours) into a logical sequence, classification, structure, or calculation.
- Connect M1.04 — Describe the relationship Among Fault Models. (2 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Introduction to VLSI Testing and Fault Models.
- Write an examination answer on M1.04 — Describe the relationship Among Fault Models. (2 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading M1.04 — Describe the relationship Among Fault Models. (2 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of M1.04 — Describe the relationship Among Fault Models. (2 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on M1.04 — Describe the relationship Among Fault Models. (2 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is M1.04 — Describe the relationship Among Fault Models. (2 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining M1.04 — Describe the relationship Among Fault Models. (2 hours)?
- How would you distinguish M1.04 — Describe the relationship Among Fault Models. (2 hours) from a closely related idea?
- Where would an engineer or technician encounter M1.04 — Describe the relationship Among Fault Models. (2 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving M1.04 — Describe the relationship Among Fault Models. (2 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: M1.04 — Describe the relationship Among Fault Models. (2 hours) □□□□ topic □□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□.

Module summary: Module 1 establishes the testing vocabulary: process, equipment, quality, abstraction, and the limits of each fault model.

OFFICIAL MODULE 2

Logic and Fault Simulation and Testability Measures

Simulation predicts circuit behaviour before or during test evaluation. True-value simulation checks the fault-free response, while fault simulation injects selected faults and observes whether test patterns expose them at measurable outputs. The module also introduces Boolean Difference, path sensitization, and Kohavi-style reasoning for identifying faults in logic circuits.

Hours: 20
CO2 · Bloom level: Understanding

Handbook study routine: Complete Module 2 in three passes. First read the official source coverage and topic headings. Next study every Handbook treatment, writing the key definition, relationship, procedure, formula, diagram, or comparison in your own words. Finally close the notes and answer the self-check questions and the module questions without looking at the answer.

Official source coverage for Module 2

Source basis: Official Contents block. Every point below is retained and linked to a study-oriented explanation. The main topic cards remain the primary lesson narrative.

View extracted official source transcript

:
LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation
Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm

Point-by-point study guide

Official syllabus point 1: LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for

Exact source point

Syllabus text: LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for

How to understand and study it

This point is an official syllabus requirement: “LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for”. Record the relevant quantity, symbol, unit, relation or formula, and the interpretation of the result. Do not memorise a number without its unit and condition. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Record the symbol, SI unit, governing relation, and one correctly labelled calculation.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

Malayalam support: □ syllabus point □□□□□□□□□□□ LOGIC & FAULT SIMULATION & TESTABILITY MEASURES, Simulation for □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

Exam focus: Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for using the official terminology retained above.
- Organise the important elements of LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for into a logical sequence, classification, structure, or calculation.
- Connect LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for with one engineering decision, operation, measurement, or safety requirement in the context of Logic and Fault Simulation and Testability Measures.
- Write an examination answer on LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for. It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for?
- How would you distinguish LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for from a closely related idea?
- Where would an engineer or technician encounter LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for, and what must be checked before using it?
- What common mistake would make an answer or operation involving LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: LOGIC & FAULT SIMULATION & TESTABILITY MEASURES: Simulation for [illegible] topic [illegible] [illegible] exact technical term [illegible]. [illegible] definition [illegible] principle, named parts/steps, application, limitation [illegible] [illegible] [illegible]. English terminology, symbols, units, diagram labels [illegible] [illegible]. Exam answer [illegible] concept-[illegible] [illegible]–[illegible] [illegible] [illegible].

Official syllabus point 2: Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation

Exact source point

Syllabus text: Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation

How to understand and study it

This point is an official syllabus requirement: “Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation”. Record the relevant quantity, symbol, unit, relation or formula, and the interpretation of the result. Do not memorise a number without its unit and condition. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Practise one small input-output example and test at least one boundary or fault condition.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

Malayalam support: □ syllabus point □□□□□□□□□□□□ Design Verification, Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value, Fault Simulation □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

Exam focus: Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation must be learned as both a concept and a calculation procedure. The student should know what each quantity represents, the conditions under which a relation is valid, the unit of every quantity, and how to check whether the final answer is physically reasonable.

Learning objectives

- Define and scope Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation using the official terminology retained above.
- Organise the important elements of Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation into a logical sequence, classification, structure, or calculation.
- Connect Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation with one engineering decision, operation, measurement, or safety requirement in the context of Logic and Fault Simulation and Testability Measures.
- Write an examination answer on Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation. It is a study organisation method, not an additional official syllabus requirement.

- Identify the given quantities and write them with symbols and SI units.
- Select the governing definition, equation, graph, or relationship instead of starting with arithmetic.
- Substitute values only after checking units and the stated operating condition.
- Interpret the result in words and check its sign, magnitude, unit, and limiting behaviour.

Engineering application lens

In an engineering setting, Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation is useful when a designer or technician must convert an observation into a measurable value, predict a response, compare alternatives, or verify that a component is operating within its rated condition. The practical question is always: what is measured or known, what is required, what model is appropriate, and how will the result be checked?

Worked answer method

Given: the quantities and conditions stated in the question.
Required: the unknown quantity, including its unit.
Calculation: write the governing relation symbolically, substitute consistently converted values, and show the unit cancellation.
Answer: report the result with a suitable number of significant figures and one sentence of interpretation.

Exam answer blueprint

For a short-answer question on Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation?
- How would you distinguish Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation from a closely related idea?
- Where would an engineer or technician encounter Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation, and what must be checked before using it?
- What common mistake would make an answer or operation involving Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation incorrect?

Common mistakes and safety emphasis

- Using a memorised relation without checking its conditions.

- Mixing prefixes or units, such as milli, kilo, mega, seconds, minutes, or degrees.
- Reporting a numerical value without a unit or without explaining what it means.
- Rounding intermediate values so aggressively that the final answer changes.

Malayalam support: Design Verification and Test Evaluation – Modeling Circuits for Simulation – Algorithms for True Value and Fault Simulation □□□□ topic □□□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□□ □□□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□□. Exam answer □□□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□□.

Official syllabus point 3: Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm

Exact source point

Syllabus text: Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm

How to understand and study it

This point is an official syllabus requirement: “Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm”. Prepare a comparison using the same criteria for every item: principle, performance, cost or complexity, limitation, and application. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Practise one small input-output example and test at least one boundary or fault condition.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

Malayalam support: □ syllabus point □□□□□□□□□□□□□ Fault Modelling, Boolean Difference Method, Path sensitization method, Kohavi Algorithm □□□□ technical terms English-□□□□□□□□ □□□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

Exam focus: Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm using the official terminology retained above.
- Organise the important elements of Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm into a logical sequence, classification, structure, or calculation.
- Connect Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm with one engineering decision, operation, measurement, or safety requirement in the context of Logic and Fault Simulation and Testability Measures.
- Write an examination answer on Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm. It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm?
- How would you distinguish Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm from a closely related idea?
- Where would an engineer or technician encounter Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm, and what must be checked before using it?
- What common mistake would make an answer or operation involving Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

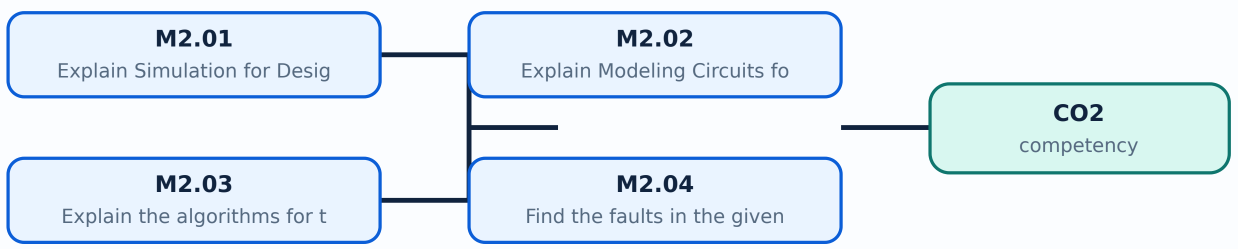
Malayalam support: Fault Modelling: Boolean Difference Method, Path sensitization method, Kohavi Algorithm □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□.

Learning goals

- Explain the purpose of simulation for design verification and test evaluation.
- Model logic circuits so that good and faulty responses can be compared.
- Apply the idea of fault activation and propagation to identify detected faults.

Module study tip

Read every topic in sequence, reproduce its example or procedure, and write a short explanation before attempting the module questions.



Learning map for Module module-2: the listed topics build the module competency.

M2.01 — Explain Simulation for Design Verification and Test Evaluation (3 hours)

Concept and explanation

Simulation applies input vectors to a circuit model and computes the resulting internal and output values. In design verification, the objective is to check whether the implementation follows the specification. In test evaluation, the objective is to determine whether a test pattern detects a targeted fault. A good simulation flow defines inputs and clocking, establishes expected responses, runs the fault-free model, introduces faults when required, and records mismatches, coverage, and unresolved cases.

> **Malayalam support:** Simulation input vectors circuit-[illegible] [illegible] output response [illegible]. Design verification specification [illegible] [illegible] [illegible]; test evaluation test pattern fault [illegible] [illegible] [illegible]. Good model-[illegible] faulty model-[illegible] compare [illegible].

Study points

- Separate fault-free simulation from fault simulation.
- Describe stimulus, response, comparison, and coverage as parts of a simulation flow.
- Explain why a mismatch must be traced to an input, model, timing assumption, or fault.

> **Engineering / workplace application:** HDL simulation, regression testing, and production-test pattern validation reduce the chance that an incorrect test sequence reaches silicon.

> **Illustrative example:** For input vector 10, simulate the good circuit and a line stuck-at-1 circuit; if the output differs, the vector detects that fault.

> **Common mistake:** A waveform that looks different is not automatically a detected fault unless the fault-free expected response and observation point are defined.

Handbook treatment

M2.01 — Explain Simulation for Design Verification and Test Evaluation (3 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope M2.01 — Explain Simulation for Design Verification and Test Evaluation (3 hours) using the official terminology retained above.
- Organise the important elements of M2.01 — Explain Simulation for Design Verification and Test Evaluation (3 hours) into a logical sequence, classification, structure, or calculation.
- Connect M2.01 — Explain Simulation for Design Verification and Test Evaluation (3 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Logic and Fault Simulation and Testability Measures.
- Write an examination answer on M2.01 — Explain Simulation for Design Verification and Test Evaluation (3 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading M2.01 — Explain Simulation for Design Verification and Test Evaluation (3 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of M2.01 — Explain Simulation for Design Verification and Test Evaluation (3 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on M2.01 — Explain Simulation for Design Verification and Test Evaluation (3 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is M2.01 — Explain Simulation for Design Verification and Test Evaluation (3 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining M2.01 — Explain Simulation for Design Verification and Test Evaluation (3 hours)?
- How would you distinguish M2.01 — Explain Simulation for Design Verification and Test Evaluation (3 hours) from a closely related idea?
- Where would an engineer or technician encounter M2.01 — Explain Simulation for Design Verification and Test Evaluation (3 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving M2.01 — Explain Simulation for Design Verification and Test Evaluation (3 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: M2.01 — Explain Simulation for Design Verification and Test Evaluation (3 hours) □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

M2.02 — Explain Modeling Circuits for Simulation (4 hours)

Concept and explanation

A simulation model represents gates, interconnections, storage elements, delays, and external signals at a selected abstraction level. A combinational model maps current inputs to outputs; a sequential model also represents state, clock events, reset, and feedback. A useful test model has clear signal names, deterministic initial conditions, correct logic polarity, and a testbench that applies meaningful patterns. Modelling resolution matters because a gate-level model, switch-level model, and behavioural model expose different details and computational costs.

Malayalam support: Simulation model circuit-□□□□ gates, wires, storage, delays, inputs □□□□□□ represent □□□□□□□□□□□. Combinational model current inputs □□□□□□ □□□□□□□□□□□□; sequential model state, clock, reset □□□□□□□□□ □□□□□□□□□□□□□□□. Abstraction level □□□□□□□□□□ detail □□□□□.

Study points

- Choose a model level appropriate to the verification or testing question.
- Include state initialisation, clocking, reset, and signal polarity in a sequential model.
- Explain why model assumptions must be documented.

Engineering / workplace application: Reusable gate models and testbenches support regression testing, fault injection, and comparison of alternative implementations.

Illustrative example: A simple combinational testbench may cycle through all four inputs of a two-input gate, whereas a sequential testbench must also define reset and clock transitions.

Common mistake: Leaving an uninitialised sequential state undocumented can produce X values or misleading fault conclusions.

Handbook treatment

M2.02 — Explain Modeling Circuits for Simulation (4 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope M2.02 — Explain Modeling Circuits for Simulation (4 hours) using the official terminology retained above.
- Organise the important elements of M2.02 — Explain Modeling Circuits for Simulation (4 hours) into a logical sequence, classification, structure, or calculation.
- Connect M2.02 — Explain Modeling Circuits for Simulation (4 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Logic and Fault Simulation and Testability Measures.
- Write an examination answer on M2.02 — Explain Modeling Circuits for Simulation (4 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading M2.02 — Explain Modeling Circuits for Simulation (4 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of M2.02 — Explain Modeling Circuits for Simulation (4 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on M2.02 — Explain Modeling Circuits for Simulation (4 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is M2.02 — Explain Modeling Circuits for Simulation (4 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining M2.02 — Explain Modeling Circuits for Simulation (4 hours)?
- How would you distinguish M2.02 — Explain Modeling Circuits for Simulation (4 hours) from a closely related idea?
- Where would an engineer or technician encounter M2.02 — Explain Modeling Circuits for Simulation (4 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving M2.02 — Explain Modeling Circuits for Simulation (4 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: M2.02 — Explain Modeling Circuits for Simulation (4 hours)
□□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□.

M2.03 — Explain the algorithms for true value and fault simulation (6 hours)

Concept and explanation

True-value simulation evaluates the fault-free circuit for each input vector. Fault simulation repeats or augments that computation for circuits containing targeted faults. A fault is activated when the good value at its site differs from the faulty value; it is propagated when a sensitised path carries that difference to a primary output or scan observation point. Fault lists can be reduced using equivalence or dominance assumptions, and detected faults can be dropped from later simulation. The Boolean Difference Method relates output sensitivity to an input or internal variable; path sensitization selects non-controlling values on side inputs so the fault effect can pass through a gate; the Kohavi Algorithm is an algorithmic procedure for logic-fault analysis and test generation.

Malayalam support: Good circuit value□□□ faulty circuit value□□□ compare □□□□□□□□□□□□□ fault simulation. Fault site-□ values □□□□□□□□□□□□□□□□□□ activation; output □□□ difference □□□□□□□□□□ propagation. Boolean Difference sensitivity □□□□□□□□□; path sensitization side inputs non-controlling values □□□□□□□□□.

Study points

- Define activation and propagation.
- Explain fault dropping, equivalence, and dominance as ways to reduce computation.
- State the roles of Boolean Difference Method, path sensitization method, and Kohavi Algorithm.

Engineering / workplace application: ATPG and fault-simulation tools use these ideas to evaluate large fault lists without manually analysing every transistor.

Illustrative example: For an AND gate, a fault effect on one input can propagate only when the other input is at the non-controlling value 1; a 0 on the other input masks the difference.

Common mistake: Activating a fault is not sufficient; the effect must also propagate to an observable point.

Handbook treatment

M2.03 — Explain the algorithms for true value and fault simulation (6 hours) must be learned as both a concept and a calculation procedure. The student should know what each quantity represents, the conditions under which a relation is valid, the unit of every quantity, and how to check whether the final answer is physically reasonable.

Learning objectives

- Define and scope M2.03 — Explain the algorithms for true value and fault simulation (6 hours) using the official terminology retained above.
- Organise the important elements of M2.03 — Explain the algorithms for true value and fault simulation (6 hours) into a logical sequence, classification, structure, or calculation.
- Connect M2.03 — Explain the algorithms for true value and fault simulation (6 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Logic and Fault Simulation and Testability Measures.
- Write an examination answer on M2.03 — Explain the algorithms for true value and fault simulation (6 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading M2.03 — Explain the algorithms for true value and fault simulation (6 hours). It is a study organisation method, not an additional official syllabus requirement.

- Identify the given quantities and write them with symbols and SI units.
- Select the governing definition, equation, graph, or relationship instead of starting with arithmetic.
- Substitute values only after checking units and the stated operating condition.
- Interpret the result in words and check its sign, magnitude, unit, and limiting behaviour.

Engineering application lens

In an engineering setting, M2.03 — Explain the algorithms for true value and fault simulation (6 hours) is useful when a designer or technician must convert an observation into a measurable value, predict a response, compare alternatives, or verify that a component is operating within its rated condition. The practical question is always: what is measured or known, what is required, what model is appropriate, and how will the result be checked?

Worked answer method

Given: the quantities and conditions stated in the question.
Required: the unknown quantity, including its unit.
Calculation: write the governing relation symbolically, substitute consistently converted values, and show the unit cancellation.
Answer: report the result with a suitable number of significant figures and one sentence of interpretation.

Exam answer blueprint

For a short-answer question on M2.03 — Explain the algorithms for true value and fault simulation (6 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is M2.03 — Explain the algorithms for true value and fault simulation (6 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining M2.03 — Explain the algorithms for true value and fault simulation (6 hours)?
- How would you distinguish M2.03 — Explain the algorithms for true value and fault simulation (6 hours) from a closely related idea?
- Where would an engineer or technician encounter M2.03 — Explain the algorithms for true value and fault simulation (6 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving M2.03 — Explain the algorithms for true value and fault simulation (6 hours) incorrect?

Common mistakes and safety emphasis

- Using a memorised relation without checking its conditions.
- Mixing prefixes or units, such as milli, kilo, mega, seconds, minutes, or degrees.
- Reporting a numerical value without a unit or without explaining what it means.
- Rounding intermediate values so aggressively that the final answer changes.

Malayalam support: M2.03 — Explain the algorithms for true value and fault simulation (6 hours) □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□.

M2.04 — Find the faults in the given logic circuit (7 hours)

Concept and explanation

Finding faults in a logic circuit follows a repeatable sequence. Identify the fault site and fault type, determine the good value required to activate it, choose values on side inputs that avoid masking, propagate the difference through a path, and observe the output. If no path can be sensitised, the fault may be redundant for the chosen observation or equivalent to another fault. For sequential logic, the procedure also includes state initialisation, clock sequencing, and observation over time. A fault table records the vector, expected good output, faulty output, and detection status.

Malayalam support: Circuit-□ fault □□□□□□□□□□ □□□□□ fault activate □□□□□□□, □□□□□□ side inputs □□□□□□□□□□ effect output □□□ propagate □□□□□□□, □□□□□□ output observe □□□□□□□. Sequential circuit-□ state, clock sequence □□□□□□□□□ □□□□□□□□□□□□□□□□□.

Study points

- Construct a good/faulty response table for a small combinational circuit.
- Use controlling and non-controlling values to reason about propagation.
- Record detected, undetected, redundant, and possibly equivalent faults separately.

Engineering / workplace application: Fault tables are useful in laboratory exercises, ATPG debugging, design-for-test reviews, and failure-analysis reports.

Illustrative example: For a two-level AND-OR network, select a vector that makes the target branch active and every side input on the propagation path non-controlling, then compare the final output.

Common mistake: Skipping the good-circuit response makes it impossible to prove that the observed output difference is caused by the injected fault.

Handbook treatment

M2.04 — Find the faults in the given logic circuit (7 hours) should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope M2.04 — Find the faults in the given logic circuit (7 hours) using the official terminology retained above.
- Organise the important elements of M2.04 — Find the faults in the given logic circuit (7 hours) into a logical sequence, classification, structure, or calculation.
- Connect M2.04 — Find the faults in the given logic circuit (7 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Logic and Fault Simulation and Testability Measures.
- Write an examination answer on M2.04 — Find the faults in the given logic circuit (7 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading M2.04 — Find the faults in the given logic circuit (7 hours). It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, M2.04 — Find the faults in the given logic circuit (7 hours) matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on M2.04 — Find the faults in the given logic circuit (7 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is M2.04 — Find the faults in the given logic circuit (7 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining M2.04 — Find the faults in the given logic circuit (7 hours)?
- How would you distinguish M2.04 — Find the faults in the given logic circuit (7 hours) from a closely related idea?
- Where would an engineer or technician encounter M2.04 — Find the faults in the given logic circuit (7 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving M2.04 — Find the faults in the given logic circuit (7 hours) incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: M2.04 — Find the faults in the given logic circuit (7 hours) □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□.

Module summary: Module 2 turns fault models into observable evidence through good-value simulation, faulty-value simulation, activation, propagation, and testability reasoning.

OFFICIAL MODULE 3

Test Generation for Combinational and Sequential Circuits

Automatic Test Pattern Generation (ATPG) searches for input or input-sequence patterns that detect specified faults. The search must represent circuit values and fault conditions, identify redundant logic, and handle the difference between memoryless combinational logic and state-dependent sequential logic.

Hours: 14
CO3 · Bloom level: Understanding

Handbook study routine: Complete Module 3 in three passes. First read the official source coverage and topic headings. Next study every Handbook treatment, writing the key definition, relationship, procedure, formula, diagram, or comparison in your own words. Finally close the notes and answer the self-check questions and the module questions without looking at the answer.

Official source coverage for Module 3

Source basis: Official Contents block. Every point below is retained and linked to a study-oriented explanation. The main topic cards remain the primary lesson narrative.

View extracted official source transcript

:
TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS
Algorithms and Representations – Redundancy Identification – Combinational ATPG Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm
Based ATPG

Point-by-point study guide

Official syllabus point 1: TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS

Exact source point

> **Syllabus text:** TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS

How to understand and study it

This point is an official syllabus requirement: “TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS”. Treat every named term as an examinable subpoint. Define it, explain its role or behaviour, distinguish it from related terms, and connect it to the module application. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. List the named terms separately and add one distinguishing feature or engineering use for each.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

> **Malayalam support:** □ syllabus point □□□□□□□□□□□ TEST GENERATION FOR COMBINATIONAL, SEQUENTIAL CIRCUITS □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

> **Exam focus:** Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS using the official terminology retained above.
- Organise the important elements of TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS into a logical sequence, classification, structure, or calculation.
- Connect TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS with one engineering decision, operation, measurement, or safety requirement in the context of Test Generation for Combinational and Sequential Circuits.
- Write an examination answer on TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS. It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS?
- How would you distinguish TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS from a closely related idea?
- Where would an engineer or technician encounter TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS, and what must be checked before using it?
- What common mistake would make an answer or operation involving TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: TEST GENERATION FOR COMBINATIONAL AND SEQUENTIAL CIRCUITS [illegible] topic [illegible] [illegible] exact technical term [illegible]. [illegible] definition [illegible] principle, named parts/steps, application, limitation [illegible] [illegible] [illegible]. English terminology, symbols, units, diagram labels [illegible] [illegible]. Exam answer [illegible] concept-[illegible] [illegible]–[illegible] [illegible] [illegible].

Official syllabus point 2: Algorithms and Representations – Redundancy Identification – Combinational ATPG

Exact source point

Syllabus text: Algorithms and Representations – Redundancy Identification – Combinational ATPG

How to understand and study it

This point is an official syllabus requirement: “Algorithms and Representations – Redundancy Identification – Combinational ATPG”. Study the input, logic or algorithm, output, and one test condition. Write the steps in order and identify the common error that changes the result. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Practise one small input-output example and test at least one boundary or fault condition.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

Malayalam support: ▯ syllabus point ▯▯▯▯▯▯▯▯▯▯▯▯ Algorithms, Representations – Redundancy Identification – Combinational ATPG ▯▯▯▯ technical terms English-▯▯▯▯▯▯▯ ▯▯▯▯▯▯▯. ▯▯▯▯▯ definition ▯▯▯▯▯▯▯▯▯▯ principle ▯▯▯▯▯▯▯▯▯▯▯▯▯; ▯▯▯▯ ▯▯▯ term-▯▯▯▯ function, difference, application ▯▯▯▯▯▯ ▯▯▯▯▯▯▯▯▯▯ ▯▯▯▯▯▯. Diagram, sequence, formula, unit ▯▯▯▯▯▯ ▯▯▯▯▯▯▯▯▯▯ ▯▯▯▯▯ ▯▯▯▯▯▯▯ revision ▯▯▯▯▯▯▯.

Exam focus: Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

Algorithms and Representations – Redundancy Identification – Combinational ATPG should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope Algorithms and Representations – Redundancy Identification – Combinational ATPG using the official terminology retained above.
- Organise the important elements of Algorithms and Representations – Redundancy Identification – Combinational ATPG into a logical sequence, classification, structure, or calculation.
- Connect Algorithms and Representations – Redundancy Identification – Combinational ATPG with one engineering decision, operation, measurement, or safety requirement in the context of Test Generation for Combinational and Sequential Circuits.
- Write an examination answer on Algorithms and Representations – Redundancy Identification – Combinational ATPG with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading Algorithms and Representations – Redundancy Identification – Combinational ATPG. It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, Algorithms and Representations – Redundancy Identification – Combinational ATPG matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on Algorithms and Representations – Redundancy Identification – Combinational ATPG, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is Algorithms and Representations – Redundancy Identification – Combinational ATPG, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining Algorithms and Representations – Redundancy Identification – Combinational ATPG?
- How would you distinguish Algorithms and Representations – Redundancy Identification – Combinational ATPG from a closely related idea?
- Where would an engineer or technician encounter Algorithms and Representations – Redundancy Identification – Combinational ATPG, and what must be checked before using it?
- What common mistake would make an answer or operation involving Algorithms and Representations – Redundancy Identification – Combinational ATPG incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: Algorithms and Representations – Redundancy Identification – Combinational ATPG [illegible] topic [illegible] [illegible] exact technical term [illegible]. [illegible] definition [illegible] principle, named parts/steps, application, limitation [illegible] [illegible] [illegible]. English terminology, symbols, units, diagram labels [illegible] [illegible]. Exam answer [illegible] concept-[illegible] [illegible]–[illegible] [illegible] [illegible].

Official syllabus point 3: Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm

Exact source point

Syllabus text: Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm

How to understand and study it

This point is an official syllabus requirement: “Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm”. Study the input, logic or algorithm, output, and one test condition. Write the steps in order and identify the common error that changes the result. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Practise one small input-output example and test at least one boundary or fault condition.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

Malayalam support: □ syllabus point □□□□□□□□□□□ Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

Exam focus: Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm using the official terminology retained above.
- Organise the important elements of Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm into a logical sequence, classification, structure, or calculation.
- Connect Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm with one engineering decision, operation, measurement, or safety requirement in the context of Test Generation for Combinational and Sequential Circuits.
- Write an examination answer on Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm. It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm?
- How would you distinguish Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm from a closely related idea?
- Where would an engineer or technician encounter Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm, and what must be checked before using it?
- What common mistake would make an answer or operation involving Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: Algorithms – Sequential ATPG Algorithms – Simulation Based ATPG – Genetic Algorithm [illegible] topic [illegible] [illegible] exact technical term [illegible]. [illegible] definition [illegible] principle, named parts/steps, application, limitation [illegible] [illegible] [illegible]. English terminology, symbols, units, diagram labels [illegible] [illegible]. Exam answer [illegible] concept-[illegible] [illegible]–[illegible] [illegible] [illegible].

Official syllabus point 4: Based ATPG

Exact source point

Syllabus text: Based ATPG

How to understand and study it

This point is an official syllabus requirement: “Based ATPG”. Treat every named term as an examinable subpoint. Define it, explain its role or behaviour, distinguish it from related terms, and connect it to the module application. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. List the named terms separately and add one distinguishing feature or engineering use for each.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

Malayalam support: □ syllabus point □□□□□□□□□□□□ Based ATPG □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

Exam focus: Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

Based ATPG is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope Based ATPG using the official terminology retained above.
- Organise the important elements of Based ATPG into a logical sequence, classification, structure, or calculation.
- Connect Based ATPG with one engineering decision, operation, measurement, or safety requirement in the context of Test Generation for Combinational and Sequential Circuits.
- Write an examination answer on Based ATPG with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading Based ATPG. It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of Based ATPG is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on Based ATPG, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is Based ATPG, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining Based ATPG?
- How would you distinguish Based ATPG from a closely related idea?
- Where would an engineer or technician encounter Based ATPG, and what must be checked before using it?
- What common mistake would make an answer or operation involving Based ATPG incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

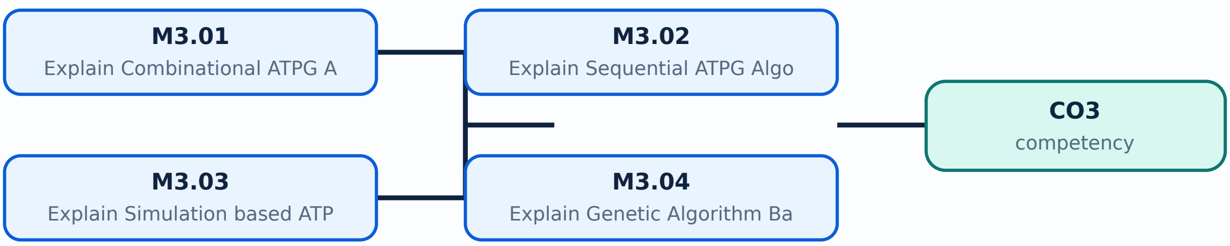
Malayalam support: Based ATPG □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

Learning goals

- Describe the representations and objectives used by ATPG algorithms.
- Differentiate combinational and sequential test-generation problems.
- Explain simulation-based and genetic-algorithm-based approaches at a conceptual level.

Module study tip

Read every topic in sequence, reproduce its example or procedure, and write a short explanation before attempting the module questions.



Learning map for Module module-3: the listed topics build the module competency.

M3.01 — Explain Combinational ATPG Algorithms (3 hours)

Concept and explanation

Combinational ATPG finds an input vector that activates a fault and propagates its effect to a primary output. Common reasoning steps are fault activation, forward implication, backward implication, path sensitization, and conflict resolution. Algorithms may use circuit topology, logic values, D-algebra or related symbolic representations, and systematic backtracking. A valid result must satisfy both the activation condition and the observation condition; if every possible path is blocked, the fault is redundant or untestable under the model.

Malayalam support: Combinational ATPG-□ input vector □□□□□□□□□ fault activate □□□□□□□□□□ output □□□ propagate □□□□□□□□□□ □□□□. Forward implication, backward implication, path sensitization, conflict resolution □□□□□□ □□□□□□ steps □□□.

Study points

- State the two essential conditions for detecting a combinational fault.
- Explain the purpose of implication and backtracking.
- Identify redundant or untestable faults conceptually.

Engineering / workplace application: Combinational ATPG creates production test patterns for logic blocks, arithmetic units, control logic, and scan-test combinational portions.

Illustrative example: If a target line must be 1 for activation and a downstream OR gate needs its other input 0 for propagation, ATPG assigns both conditions before checking upstream consistency.

Common mistake: A partial assignment that satisfies activation but causes a propagation conflict is not a completed ATPG solution.

Handbook treatment

M3.01 — Explain Combinational ATPG Algorithms (3 hours) should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope M3.01 — Explain Combinational ATPG Algorithms (3 hours) using the official terminology retained above.
- Organise the important elements of M3.01 — Explain Combinational ATPG Algorithms (3 hours) into a logical sequence, classification, structure, or calculation.
- Connect M3.01 — Explain Combinational ATPG Algorithms (3 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Test Generation for Combinational and Sequential Circuits.
- Write an examination answer on M3.01 — Explain Combinational ATPG Algorithms (3 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading M3.01 — Explain Combinational ATPG Algorithms (3 hours). It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, M3.01 — Explain Combinational ATPG Algorithms (3 hours) matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on M3.01 — Explain Combinational ATPG Algorithms (3 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is M3.01 — Explain Combinational ATPG Algorithms (3 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining M3.01 — Explain Combinational ATPG Algorithms (3 hours)?
- How would you distinguish M3.01 — Explain Combinational ATPG Algorithms (3 hours) from a closely related idea?
- Where would an engineer or technician encounter M3.01 — Explain Combinational ATPG Algorithms (3 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving M3.01 — Explain Combinational ATPG Algorithms (3 hours) incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: M3.01 — Explain Combinational ATPG Algorithms (3 hours) [illegible] topic [illegible] [illegible] exact technical term [illegible]. [illegible] definition [illegible] principle, named parts/steps, application, limitation [illegible] [illegible] [illegible]. English terminology, symbols, units, diagram labels [illegible] [illegible]. Exam answer [illegible] concept-[illegible] [illegible]–[illegible] [illegible] [illegible].

M3.02 — Explain Sequential ATPG Algorithms (3 hours)

Concept and explanation

Sequential ATPG must control and observe circuit state as well as primary inputs. A test sequence may require an initialisation or reset operation, several capture cycles, and a final observation cycle. The algorithm searches through state transitions, uses time-frame expansion or state justification, and accounts for feedback and storage elements. Scan design simplifies the problem by making internal state directly loadable and observable; without scan, the required sequence may be longer and the search space much larger.

> **Malayalam support:** Sequential ATPG-□ input □□□□□□ □□□□□□□; internal state initialize □□□□□□□□□□ clock cycles □□□ fault effect propagate □□□□□□□□□□ □□□□. Scan design state load/observe □□□□□□□□□□□ □□□□□□□□□□□□□□□□.

Study points

- Explain why a sequential test is a sequence rather than one vector.
- Include reset, state justification, capture, and observation in the reasoning.
- Compare the difficulty of scan and non-scan sequential test generation.

> **Engineering / workplace application:** Microcontrollers, controllers, pipelines, and finite-state machines use sequential ATPG or scan-assisted ATPG to detect storage and transition-related defects.

> **Illustrative example:** A sequence may reset a state machine, drive it to a target state, apply a sensitising input, and capture the faulty transition at the next clock edge.

> **Common mistake:** Applying a combinational ATPG vector at one instant does not prove that a sequential fault is detected across clocked state transitions.

Handbook treatment

M3.02 — Explain Sequential ATPG Algorithms (3 hours) should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope M3.02 — Explain Sequential ATPG Algorithms (3 hours) using the official terminology retained above.
- Organise the important elements of M3.02 — Explain Sequential ATPG Algorithms (3 hours) into a logical sequence, classification, structure, or calculation.
- Connect M3.02 — Explain Sequential ATPG Algorithms (3 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Test Generation for Combinational and Sequential Circuits.
- Write an examination answer on M3.02 — Explain Sequential ATPG Algorithms (3 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading M3.02 — Explain Sequential ATPG Algorithms (3 hours). It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, M3.02 — Explain Sequential ATPG Algorithms (3 hours) matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on M3.02 — Explain Sequential ATPG Algorithms (3 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is M3.02 — Explain Sequential ATPG Algorithms (3 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining M3.02 — Explain Sequential ATPG Algorithms (3 hours)?
- How would you distinguish M3.02 — Explain Sequential ATPG Algorithms (3 hours) from a closely related idea?
- Where would an engineer or technician encounter M3.02 — Explain Sequential ATPG Algorithms (3 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving M3.02 — Explain Sequential ATPG Algorithms (3 hours) incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: M3.02 — Explain Sequential ATPG Algorithms (3 hours)
[illegible] topic [illegible] [illegible] exact technical term [illegible]. [illegible] definition [illegible] principle, named parts/steps, application, limitation [illegible] [illegible] [illegible]. English terminology, symbols, units, diagram labels [illegible] [illegible]. Exam answer [illegible] concept-[illegible] [illegible]–[illegible] [illegible] [illegible].

M3.03 — Explain Simulation based ATPG (4 hours)

Concept and explanation

Simulation-based ATPG uses circuit simulation to evaluate candidate test patterns and guide the search toward patterns that detect remaining faults. It may begin with random or deterministic patterns, simulate the fault list, drop detected faults, and generate additional patterns for faults that remain undetected. The method is simple to integrate with existing simulators and can quickly detect easy faults, but it may require many patterns for hard-to-detect faults and does not guarantee a minimum test set.

Malayalam support: Simulation-based ATPG candidate patterns simulate □□□□□□ detected faults remove □□□□□□□□□□. Remaining hard faults-□□□□□ □□□□□ patterns □□□□□□□□□□□□□□. Easy faults □□□□□□□□ □□□□□□□□□□□; hard faults-□□ □□□□□□ patterns □□□□□□□□□□.

Study points

- Describe fault dropping in a simulation-based ATPG loop.
- Compare random stimulation with deterministic pattern generation.
- State the trade-off between simplicity, pattern count, and guaranteed coverage.

Engineering / workplace application: Simulation-based pattern generation is useful for early verification, fault-list screening, and supplementing deterministic ATPG patterns.

Illustrative example: After simulating a batch of random vectors, the tool removes detected stuck-at faults and focuses deterministic search on the remaining list.

Common mistake: A large number of random patterns does not automatically mean high coverage of rare or sequentially difficult faults.

Handbook treatment

M3.03 — Explain Simulation based ATPG (4 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope M3.03 — Explain Simulation based ATPG (4 hours) using the official terminology retained above.
- Organise the important elements of M3.03 — Explain Simulation based ATPG (4 hours) into a logical sequence, classification, structure, or calculation.
- Connect M3.03 — Explain Simulation based ATPG (4 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Test Generation for Combinational and Sequential Circuits.
- Write an examination answer on M3.03 — Explain Simulation based ATPG (4 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading M3.03 — Explain Simulation based ATPG (4 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of M3.03 — Explain Simulation based ATPG (4 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on M3.03 — Explain Simulation based ATPG (4 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is M3.03 — Explain Simulation based ATPG (4 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining M3.03 — Explain Simulation based ATPG (4 hours)?
- How would you distinguish M3.03 — Explain Simulation based ATPG (4 hours) from a closely related idea?
- Where would an engineer or technician encounter M3.03 — Explain Simulation based ATPG (4 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving M3.03 — Explain Simulation based ATPG (4 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: M3.03 — Explain Simulation based ATPG (4 hours) [illegible] topic [illegible] [illegible] exact technical term [illegible]. [illegible] definition [illegible] principle, named parts/steps, application, limitation [illegible] [illegible] [illegible]. English terminology, symbols, units, diagram labels [illegible] [illegible]. Exam answer [illegible] concept-[illegible] [illegible]–[illegible] [illegible] [illegible].

M3.04 — Explain Genetic Algorithm Based ATPG (4 hours)

Concept and explanation

A Genetic Algorithm (GA) represents a candidate test pattern or sequence as a chromosome. A population of candidates is evaluated by a fitness function such as the number of faults detected, fault-effect propagation, or coverage gain. Selection retains stronger candidates; crossover combines portions of parent patterns; mutation introduces variation. The process repeats until a coverage target, iteration limit, or acceptable fitness is reached. GA-based ATPG can explore a large search space, but it is heuristic and needs careful encoding, fitness design, and reproducible stopping criteria.

Malayalam support: GA-based ATPG-□ test pattern chromosome □□□ represent □□□□□□□. Fitness function detected faults/coverage □□□□□□□. Selection, crossover, mutation □□□ □□□□□ patterns □□□□□□□□□□ search □□□□□□□□□. □□□ heuristic method □□□.

Study points

- Identify chromosome, population, fitness, selection, crossover, and mutation.
- Explain why the fitness function must reward both activation and propagation.
- State that a heuristic result is not automatically optimal or complete.

Engineering / workplace application: Heuristic ATPG is relevant when exact search is expensive, especially for complex logic networks or long sequential sequences.

Illustrative example: If a chromosome encodes eight input bits, crossover can exchange a high-order segment between two parent patterns and mutation can flip one selected bit before resimulation.

Common mistake: Using only output switching as the fitness function may produce active patterns that never expose the target fault at an observation point.

Handbook treatment

M3.04 — Explain Genetic Algorithm Based ATPG (4 hours) should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope M3.04 — Explain Genetic Algorithm Based ATPG (4 hours) using the official terminology retained above.
- Organise the important elements of M3.04 — Explain Genetic Algorithm Based ATPG (4 hours) into a logical sequence, classification, structure, or calculation.
- Connect M3.04 — Explain Genetic Algorithm Based ATPG (4 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Test Generation for Combinational and Sequential Circuits.
- Write an examination answer on M3.04 — Explain Genetic Algorithm Based ATPG (4 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading M3.04 — Explain Genetic Algorithm Based ATPG (4 hours). It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, M3.04 — Explain Genetic Algorithm Based ATPG (4 hours) matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on M3.04 — Explain Genetic Algorithm Based ATPG (4 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is M3.04 — Explain Genetic Algorithm Based ATPG (4 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining M3.04 — Explain Genetic Algorithm Based ATPG (4 hours)?
- How would you distinguish M3.04 — Explain Genetic Algorithm Based ATPG (4 hours) from a closely related idea?
- Where would an engineer or technician encounter M3.04 — Explain Genetic Algorithm Based ATPG (4 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving M3.04 — Explain Genetic Algorithm Based ATPG (4 hours) incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: M3.04 — Explain Genetic Algorithm Based ATPG (4 hours) □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

Module summary: Module 3 compares systematic and heuristic test-generation approaches for memoryless and stateful digital circuits.

OFFICIAL MODULE 4

Design for Testability, BIST and Fault Diagnosis

Design for Testability (DFT) adds structures and design practices that make internal nodes easier to control and observe. Scan chains and Built-In Self-Test (BIST) reduce the difficulty of applying external patterns, while diagnosis uses response information to locate likely fault sites rather than merely declaring pass or fail.

Hours: 13
CO4 · Bloom level: Understanding

Handbook study routine: Complete Module 4 in three passes. First read the official source coverage and topic headings. Next study every Handbook treatment, writing the key definition, relationship, procedure, formula, diagram, or comparison in your own words. Finally close the notes and answer the self-check questions and the module questions without looking at the answer.

Official source coverage for Module 4

Source basis: Official Contents block. Every point below is retained and linked to a study-oriented explanation. The main topic cards remain the primary lesson narrative.

View extracted official source transcript

:
DESIGN FOR TESTABILITY: Design for Testability Basics – Testability Analysis - Scan Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist
FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis – Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis
– Logic BIST Diagnosis.

Point-by-point study guide

Official syllabus point 1: DESIGN FOR TESTABILITY: Design for Testability Basics - Testability Analysis - Scan

Exact source point

Syllabus text: DESIGN FOR TESTABILITY: Design for Testability Basics – Testability Analysis - Scan

How to understand and study it

This point is an official syllabus requirement: “DESIGN FOR TESTABILITY: Design for Testability Basics – Testability Analysis - Scan”. Treat every named term as an examinable subpoint. Define it, explain its role or behaviour, distinguish it from related terms, and connect it to the module application. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. List the named terms separately and add one distinguishing feature or engineering use for each.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

Malayalam support: □ syllabus point □□□□□□□□□□□ DESIGN FOR TESTABILITY, Design for Testability Basics – Testability Analysis - Scan □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

Exam focus: Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

DESIGN FOR TESTABILITY: Design for Testability Basics – Testability Analysis - Scan is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope DESIGN FOR TESTABILITY: Design for Testability Basics – Testability Analysis - Scan using the official terminology retained above.
- Organise the important elements of DESIGN FOR TESTABILITY: Design for Testability Basics – Testability Analysis - Scan into a logical sequence, classification, structure, or calculation.
- Connect DESIGN FOR TESTABILITY: Design for Testability Basics – Testability Analysis - Scan with one engineering decision, operation, measurement, or safety requirement in the context of Design for Testability, BIST and Fault Diagnosis.
- Write an examination answer on DESIGN FOR TESTABILITY: Design for Testability Basics – Testability Analysis - Scan with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading DESIGN FOR TESTABILITY: Design for Testability Basics – Testability Analysis - Scan. It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of DESIGN FOR TESTABILITY: Design for Testability Basics – Testability Analysis - Scan is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on DESIGN FOR TESTABILITY: Design for Testability Basics – Testability Analysis - Scan, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is DESIGN FOR TESTABILITY: Design for Testability Basics – Testability Analysis - Scan, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining DESIGN FOR TESTABILITY: Design for Testability Basics – Testability Analysis - Scan?
- How would you distinguish DESIGN FOR TESTABILITY: Design for Testability Basics – Testability Analysis - Scan from a closely related idea?
- Where would an engineer or technician encounter DESIGN FOR TESTABILITY: Design for Testability Basics – Testability Analysis - Scan, and what must be checked before using it?
- What common mistake would make an answer or operation involving DESIGN FOR TESTABILITY: Design for Testability Basics – Testability Analysis - Scan incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: DESIGN FOR TESTABILITY: Design for Testability Basics – Testability Analysis - Scan [illegible] topic [illegible] [illegible] exact technical term [illegible]. [illegible] definition [illegible] principle, named parts/steps, application, limitation [illegible] [illegible] [illegible]. English terminology, symbols, units, diagram labels [illegible] [illegible]. Exam answer [illegible] concept-[illegible] [illegible]–[illegible] [illegible] [illegible].

Official syllabus point 2: Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist

Exact source point

Syllabus text: Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist

How to understand and study it

This point is an official syllabus requirement: “Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist”. Study this as a structure: identify each named part, state its function, and explain how the parts are connected or arranged. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. Draw a labelled arrangement or block diagram and write the function of every labelled part.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

Malayalam support: □ syllabus point □□□□□□□□□□□ Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

Exam focus: Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist using the official terminology retained above.
- Organise the important elements of Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist into a logical sequence, classification, structure, or calculation.
- Connect Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist with one engineering decision, operation, measurement, or safety requirement in the context of Design for Testability, BIST and Fault Diagnosis.
- Write an examination answer on Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist. It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist?
- How would you distinguish Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist from a closely related idea?
- Where would an engineer or technician encounter Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist, and what must be checked before using it?
- What common mistake would make an answer or operation involving Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: Cell Designs – Scan Architecture – Built in Self-Test – Random Logic Bist [illegible] topic [illegible] [illegible] exact technical term [illegible]. [illegible] definition [illegible] principle, named parts/steps, application, limitation [illegible] [illegible] [illegible]. English terminology, symbols, units, diagram labels [illegible] [illegible]. Exam answer [illegible] concept-[illegible] [illegible]–[illegible] [illegible] [illegible].

Official syllabus point 3: FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis –

Exact source point

Syllabus text: FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis –

How to understand and study it

This point is an official syllabus requirement: “FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis –”. Begin with the exact definition or meaning, then identify the purpose, scope, and terminology contained in the point. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. List the named terms separately and add one distinguishing feature or engineering use for each.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

Malayalam support: □ syllabus point □□□□□□□□□□□ FAULT DIAGNOSIS, Introduction, Basic Definitions – Fault Models for Diagnosis – □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□ principle □□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

Exam focus: Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis – is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis – using the official terminology retained above.
- Organise the important elements of FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis – into a logical sequence, classification, structure, or calculation.
- Connect FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis – with one engineering decision, operation, measurement, or safety requirement in the context of Design for Testability, BIST and Fault Diagnosis.
- Write an examination answer on FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis – with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis –. It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis – is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis –, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis –, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis –?
- How would you distinguish FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis – from a closely related idea?
- Where would an engineer or technician encounter FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis –, and what must be checked before using it?
- What common mistake would make an answer or operation involving FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis – incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: FAULT DIAGNOSIS: Introduction and Basic Definitions – Fault Models for Diagnosis – □□□□ topic □□□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□□ □□□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

Official syllabus point 4: Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis

Exact source point

Syllabus text: Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis

How to understand and study it

This point is an official syllabus requirement: “Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis”. Treat every named term as an examinable subpoint. Define it, explain its role or behaviour, distinguish it from related terms, and connect it to the module application. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. List the named terms separately and add one distinguishing feature or engineering use for each.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

Malayalam support: □ syllabus point □□□□□□□□□□□ Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

Exam focus: Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis using the official terminology retained above.
- Organise the important elements of Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis into a logical sequence, classification, structure, or calculation.
- Connect Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis with one engineering decision, operation, measurement, or safety requirement in the context of Design for Testability, BIST and Fault Diagnosis.
- Write an examination answer on Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis. It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis, begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis, and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis?
- How would you distinguish Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis from a closely related idea?
- Where would an engineer or technician encounter Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis, and what must be checked before using it?
- What common mistake would make an answer or operation involving Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: Generation of Vectors for Diagnosis – Combinational Logic Diagnosis - Scan Chain Diagnosis [illegible] topic [illegible] [illegible] exact technical term [illegible]. [illegible] definition [illegible] principle, named parts/steps, application, limitation [illegible] [illegible] [illegible]. English terminology, symbols, units, diagram labels [illegible] [illegible]. Exam answer [illegible] concept-[illegible] [illegible]–[illegible] [illegible] [illegible].

Official syllabus point 5: – Logic BIST Diagnosis.

Exact source point

Syllabus text: – Logic BIST Diagnosis.

How to understand and study it

This point is an official syllabus requirement: “– Logic BIST Diagnosis.”. Treat every named term as an examinable subpoint. Define it, explain its role or behaviour, distinguish it from related terms, and connect it to the module application. The main topic explanation above gives the concept context; this point-by-point section ensures that each named item is revised separately.

Study sequence

1. Write the exact technical term and a one-sentence definition in your own words.
2. List the named terms separately and add one distinguishing feature or engineering use for each.
3. Finish with one examination answer: definition, explanation, diagram/table if relevant, and application or limitation.

Malayalam support: □ syllabus point □□□□□□□□□□□□ – Logic BIST Diagnosis. □□□□ technical terms English-□□□□□□□ □□□□□□□. □□□□□ definition □□□□□□□□□□ principle □□□□□□□□□□□□□; □□□□ □□□ term-□□□□ function, difference, application □□□□□□ □□□□□□□□□□ □□□□□□. Diagram, sequence, formula, unit □□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□ revision □□□□□□□.

Exam focus: Answer this point with its definition or principle first, followed by the named features, sequence, comparison, formula, diagram, or application that the question demands.

Handbook treatment

– Logic BIST Diagnosis. should be understood as an information-flow problem. Identify the input, the rule or program that processes it, the internal state or decision, and the output. A correct explanation must show the sequence, not merely list software terms.

Learning objectives

- Define and scope – Logic BIST Diagnosis. using the official terminology retained above.
- Organise the important elements of – Logic BIST Diagnosis. into a logical sequence, classification, structure, or calculation.
- Connect – Logic BIST Diagnosis. with one engineering decision, operation, measurement, or safety requirement in the context of Design for Testability, BIST and Fault Diagnosis.
- Write an examination answer on – Logic BIST Diagnosis. with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading – Logic BIST Diagnosis.. It is a study organisation method, not an additional official syllabus requirement.

- List the input signals, data, or conditions.
- Write the logic, algorithm, instruction sequence, or protocol rule in the order it is evaluated.
- State the output and identify what changes when an input or state changes.
- Test a normal case, a boundary case, and a fault or invalid-input case.

Engineering application lens

For engineering practice, – Logic BIST Diagnosis. matters because an automated or digital system must convert inputs into repeatable outputs. The technician should be able to trace a signal or data item, locate the decision that controls the result, and identify a safe response when an input is missing or abnormal.

Worked answer method

Given: the input conditions, required output, and any stated constraints.
Required: the logic sequence, program structure, truth table, or operating result.
Method: break the problem into named states or steps; evaluate them in order; test at least one alternate condition.
Answer: show the final sequence and state the expected output for each important condition.

Exam answer blueprint

For a short-answer question on – Logic BIST Diagnosis., begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is – Logic BIST Diagnosis., and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining – Logic BIST Diagnosis.?
- How would you distinguish – Logic BIST Diagnosis. from a closely related idea?
- Where would an engineer or technician encounter – Logic BIST Diagnosis., and what must be checked before using it?
- What common mistake would make an answer or operation involving – Logic BIST Diagnosis. incorrect?

Common mistakes and safety emphasis

- Writing instructions without defining the input and output.
- Ignoring scan order, state retention, initialization, or boundary conditions.
- Confusing a logical condition with the physical signal that represents it.
- Testing only the normal case and failing to consider a fault or interlock condition.

Malayalam support: – Logic BIST Diagnosis. □□□□ topic □□□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□□ □□□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□□.

Learning goals

- Explain controllability, observability, testability analysis, and scan structures.
- Describe BIST architecture and Random Logic BIST at a block level.
- Differentiate fault detection from fault diagnosis and identify diagnostic vector roles.

Module study tip

Read every topic in sequence, reproduce its example or procedure, and write a short explanation before attempting the module questions.

Learning map for Module module-4: the listed topics build the module competency.

M4.01 — Explain Testabilty basics and its analysis (4 hours)

Concept and explanation

Testability describes how easily a circuit can be controlled from available inputs and how easily internal effects can be observed at outputs. Controllability measures the effort needed to set a node to a required logic value; observability measures the effort needed to propagate a node effect to an observation point. Testability analysis identifies hard-to-control, hard-to-observe, reconvergent, redundant, or highly sequential regions. Scan cell designs replace or augment storage elements so internal state can be shifted in and out, and scan architecture connects these cells into one or more scan chains.

> **Malayalam support:** Controllability node-□□ □□□□□□□□ 0/1 □□□□□□ □□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□. Observability internal fault effect output-□ □□□□□ □□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□. Scan chain internal flip-flops serial □□□ load/shift □□□□□□□ □□□□□□□□□□.

Study points

- Define controllability and observability.
- Explain why reconvergent paths and storage elements create testability difficulty.
- Describe scan cell and scan-chain operation at a block level.

> **Engineering / workplace application:** DFT analysis guides scan insertion, test-point selection, compression planning, and ATPG feasibility in complex SoCs.

> **Illustrative example:** A scan chain can shift a known state into internal flip-flops, apply a capture cycle, and shift the captured response out for comparison.

> **Common mistake:** High controllability alone does not guarantee high testability; the resulting fault effect must also be observable.

Handbook treatment

M4.01 — Explain Testabilty basics and its analysis (4 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope M4.01 — Explain Testabilty basics and its analysis (4 hours) using the official terminology retained above.
- Organise the important elements of M4.01 — Explain Testabilty basics and its analysis (4 hours) into a logical sequence, classification, structure, or calculation.
- Connect M4.01 — Explain Testabilty basics and its analysis (4 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Design for Testability, BIST and Fault Diagnosis.
- Write an examination answer on M4.01 — Explain Testabilty basics and its analysis (4 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading M4.01 — Explain Testabilty basics and its analysis (4 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of M4.01 — Explain Testabilty basics and its analysis (4 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on M4.01 — Explain Testabilty basics and its analysis (4 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is M4.01 — Explain Testabilty basics and its analysis (4 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining M4.01 — Explain Testabilty basics and its analysis (4 hours)?
- How would you distinguish M4.01 — Explain Testabilty basics and its analysis (4 hours) from a closely related idea?
- Where would an engineer or technician encounter M4.01 — Explain Testabilty basics and its analysis (4 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving M4.01 — Explain Testabilty basics and its analysis (4 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: M4.01 — Explain Testabilty basics and its analysis (4 hours) □□□□ topic □□□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□.

M4.02 — Explain BIST architecture (4 hours)

Concept and explanation

Built-In Self-Test (BIST) places test-pattern generation, response compaction or analysis, control, and pass/fail decision logic on or near the chip. In Random Logic BIST, a pseudo-random pattern generator stimulates logic and a signature register compacts responses; an on-chip controller sequences reset, pattern application, capture, and completion. BIST reduces dependence on external tester pins and can support field or power-on test, but it consumes area, power, design effort, and may suffer from aliasing or random-pattern-resistant faults.

Malayalam support: BIST chip-□□□□ □□□□□□ test pattern generator, response analyzer, controller, pass/fail logic □□□□□□ □□□□□□□□□□□□□□□□□. Random Logic BIST pseudo-random patterns generate □□□□□□□□□□ signature register response compact □□□□□□□□□□ □□□□□□□.

Study points

- Identify pattern generation, response compaction, control, and decision blocks.
- Explain the use of pseudo-random patterns and a signature register.
- State BIST benefits and limitations, including area, power, and aliasing.

Engineering / workplace application: Memory BIST, processor self-test, automotive electronics, and safety-critical controllers use on-chip test structures for startup or maintenance checks.

Illustrative example: A BIST controller can apply a fixed number of pseudo-random patterns, compact each response into a Multiple-Input Signature Register, and compare the final signature.

Common mistake: A compacted signature matching the expected value is strong evidence, but signature aliasing means different responses can occasionally produce the same signature.

Handbook treatment

M4.02 — Explain BIST architecture (4 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope M4.02 — Explain BIST architecture (4 hours) using the official terminology retained above.
- Organise the important elements of M4.02 — Explain BIST architecture (4 hours) into a logical sequence, classification, structure, or calculation.
- Connect M4.02 — Explain BIST architecture (4 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Design for Testability, BIST and Fault Diagnosis.
- Write an examination answer on M4.02 — Explain BIST architecture (4 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading M4.02 — Explain BIST architecture (4 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of M4.02 — Explain BIST architecture (4 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on M4.02 — Explain BIST architecture (4 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is M4.02 — Explain BIST architecture (4 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining M4.02 — Explain BIST architecture (4 hours)?
- How would you distinguish M4.02 — Explain BIST architecture (4 hours) from a closely related idea?
- Where would an engineer or technician encounter M4.02 — Explain BIST architecture (4 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving M4.02 — Explain BIST architecture (4 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: M4.02 — Explain BIST architecture (4 hours) [illegible] topic [illegible] [illegible] exact technical term [illegible]. [illegible] definition [illegible] principle, named parts/steps, application, limitation [illegible] [illegible] [illegible]. English terminology, symbols, units, diagram labels [illegible] [illegible]. Exam answer [illegible] concept-[illegible] [illegible]–[illegible] [illegible] [illegible].

M4.03 — Explain Fault Models for Diagnosis (5 hours)

Concept and explanation

Fault diagnosis goes beyond detection: it uses failing responses to identify or rank possible fault locations and types. Diagnostic fault models may include stuck-at, bridging, transition, open, or scan-chain faults according to the observed failure mechanism. Diagnosis begins with basic definitions and a candidate fault list, generates distinguishing vectors, applies or simulates those vectors, and compares response signatures. Combinational logic diagnosis uses output differences and structural reasoning; Scan Chain Diagnosis examines shift and capture behaviour; Logic BIST diagnosis uses signatures, checkpoints, or additional diagnostic patterns to narrow the suspect region.

Malayalam support: Fault detection chip faulty ▯▯▯ ▯▯▯▯▯ ▯▯▯▯▯▯▯▯; diagnosis fault ▯▯▯▯▯▯▯▯▯▯▯▯▯ ▯▯▯ ▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯ ▯▯▯▯▯▯▯▯▯▯ ▯▯▯▯▯▯▯▯▯▯▯▯▯▯. Diagnostic vectors candidate faults ▯▯▯▯▯▯ ▯▯▯▯▯▯▯▯▯▯▯▯ ▯▯▯▯▯▯▯▯▯ ▯▯▯▯▯▯▯▯▯▯. Scan chain-▯ shift/capture response ▯▯▯▯▯▯▯▯▯ ▯▯▯▯▯▯▯▯▯▯▯▯.

Study points

- Define diagnosis and distinguish it from detection.
- Explain diagnostic vector generation and candidate-fault reduction.
- Compare combinational logic, scan-chain, and Logic BIST diagnosis.

Engineering / workplace application: Failure analysis, yield improvement, service troubleshooting, and safety monitoring use diagnosis to reduce repair time and identify recurring process defects.

Illustrative example: If two candidate faults produce the same response for one vector, apply a second vector selected so that their predicted responses differ; the result eliminates one candidate.

Common mistake: A single failing output usually identifies a symptom, not a unique physical fault; multiple distinguishing patterns are often needed.

Handbook treatment

M4.03 — Explain Fault Models for Diagnosis (5 hours) is an official examinable part of the module. Study it as a connected explanation: define the central term, identify the related elements, explain their relationship, and finish with a relevant engineering use or limitation.

Learning objectives

- Define and scope M4.03 — Explain Fault Models for Diagnosis (5 hours) using the official terminology retained above.
- Organise the important elements of M4.03 — Explain Fault Models for Diagnosis (5 hours) into a logical sequence, classification, structure, or calculation.
- Connect M4.03 — Explain Fault Models for Diagnosis (5 hours) with one engineering decision, operation, measurement, or safety requirement in the context of Design for Testability, BIST and Fault Diagnosis.
- Write an examination answer on M4.03 — Explain Fault Models for Diagnosis (5 hours) with a clear opening, technical middle, and final application or limitation.

Conceptual framework

Use the following frame while reading M4.03 — Explain Fault Models for Diagnosis (5 hours). It is a study organisation method, not an additional official syllabus requirement.

- Start with the exact technical term and its scope.
- Separate the topic into named elements, stages, types, or conditions.
- Explain the relationship using cause-and-effect, sequence, comparison, or a labelled representation.
- Apply the idea to a realistic engineering situation and state one limitation or common mistake.

Engineering application lens

The engineering value of M4.03 — Explain Fault Models for Diagnosis (5 hours) is understood by asking what requirement it satisfies, what input or condition it depends on, how the output is obtained, and how a technician or designer verifies the result. Use the module context to make the application specific rather than memorising an isolated definition.

Worked answer method

Given: the topic, operating condition, diagram, data, or situation stated in the question.
Required: definition, explanation, comparison, procedure, calculation, or application as requested.
Method: organise the answer under principle, named elements, sequence or relationship, and application.
Answer: use the requested technical terms, units, labels, assumptions, and a concluding statement.

Exam answer blueprint

For a short-answer question on M4.03 — Explain Fault Models for Diagnosis (5 hours), begin with the definition or principle and include the decisive feature. For a medium-answer question, add the named elements and their functions or sequence. For a long-answer question, add a labelled diagram, comparison, formula or procedure where relevant, one application, and one limitation or safety point. Never substitute a list of headings for the explanation.

Self-check questions

- What is M4.03 — Explain Fault Models for Diagnosis (5 hours), and what is its scope in this module?
- Which elements, stages, types, quantities, or conditions must be named when explaining M4.03 — Explain Fault Models for Diagnosis (5 hours)?
- How would you distinguish M4.03 — Explain Fault Models for Diagnosis (5 hours) from a closely related idea?
- Where would an engineer or technician encounter M4.03 — Explain Fault Models for Diagnosis (5 hours), and what must be checked before using it?
- What common mistake would make an answer or operation involving M4.03 — Explain Fault Models for Diagnosis (5 hours) incorrect?

Common mistakes and safety emphasis

- Copying a definition without explaining the mechanism or purpose.
- Listing terms without showing their relationship.
- Using an example that does not match the stated topic or condition.
- Leaving out a diagram, unit, assumption, limitation, or safety point when the question requires it.

Malayalam support: M4.03 — Explain Fault Models for Diagnosis (5 hours) □□□□ topic □□□□□□□□□□□ □□□□□ exact technical term □□□□□□□□□□□□□. □□□□ definition □□□□□□□□□□ principle, named parts/steps, application, limitation □□□□□□ □□□□□□□□ □□□□□□□□. English terminology, symbols, units, diagram labels □□□□□□ □□□□□□□□□. Exam answer □□□□□□□□□□ concept-□□□□ □□□□□–□□□ □□□□□ □□□□□□□□□□□□□.

Module summary: Module 4 makes internal circuitry testable and uses structured response information to detect, localise, and diagnose faults.

Formula and Notation Bank

Formula note: The supplied syllabus is primarily procedural or conceptual and does not prescribe a compulsory numerical formula bank. Focus on the official terminology, sequences, records, and practical demonstrations listed in the modules.

Diagram Library for Rapid Revision

Module 1: Introduction to VLSI Testing and Fault Models

Open the module to view its labelled learning-map diagram.

Module 2: Logic and Fault Simulation and Testability M...

Open the module to view its labelled learning-map diagram.

Module 3: Test Generation for Combinational and Sequential Circuits

Open the module to view its labelled learning-map diagram.

Module 4: Design for Testability, BIST and Fault Diagnosis

Open the module to view its labelled learning-map diagram.

Official Activities and Practical Requirements

Official activities / self-learning

- The supplied syllabus does not provide a separate official self-learning activity list for this theory elective.

- For study support, draw one fault model or scan/BIST block diagram after each relevant topic and explain the signal path in your own words.
- Use the worked examples and module questions as revision activities; these are educational additions, not official assessment requirements.

Practical requirements / equipment

- The supplied syllabus does not prescribe separate practical equipment for this theory elective.
- For optional practice, a digital-logic simulator or HDL testbench may be used to observe fault effects; the supplied PDF does not mandate a particular software package.

Assessment Methodology

The supplied PDF does not specify a separate CIE/ESE distribution.

- The supplied PDF states “Series Tests 2” in the course-outcome block and separately lists Series Test 1 — 1 hour after Module 2 and Series Test 2 — 1 hour after Module 4.
- A separate CIE/ESE mark distribution, attendance component, and official question-paper pattern are not specified in the supplied PDF; no marks or attendance values have been invented here.
- The official module durations total 58 hours; the two separately listed one-hour series tests account for the 60 periods per semester.

Module-wise Questions and Answers

module-1 — Introduction to VLSI Testing and Fault Models

1. Explain Explain VLSI Testing Process. **[3 marks]**

The VLSI testing process converts a design specification into test requirements, derives test patterns, applies them to a device, captures the responses, compares the responses with expected values, and records pass/fail information. Testing may occur at wafer level, after packaging, during production screening, and during system-level diagnosis. A useful test flow separates defect prevention, design verification, manufacturing defect detection, quality grading, and failure analysis. Test coverage is the proportion of targeted faults detected by the applied test set; high coverage improves confidence but does not mean that every possible physical defect has been observed.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

2. Explain Explain VLSI Testing Equipment. **[3 marks]**

Automatic Test Equipment (ATE) supplies controlled input patterns, timing, power, and measurement functions to a device under test. A typical setup includes a tester controller, pattern memory, timing and signal-generation resources, power supplies, pin electronics, a load board or probe card, and response-analysis hardware. Equipment selection is influenced by pin count, operating frequency, voltage levels, analogue or mixed-signal content, memory depth, measurement accuracy, and cost. The equipment must control signal integrity and protect the device while collecting repeatable results.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

3. Explain Explain fault models. **[3 marks]**

A fault model is a simplified logical representation of a physical defect. Common digital models include stuck-at-0 and stuck-at-1 faults, open faults, short or bridging faults, transistor-level defects, and delay faults. The stuck-at model assumes that a signal line remains permanently at one logic value even when the fault-free circuit would produce the other value. A model is useful when it preserves the defect behaviours that matter for test generation without requiring every transistor-level detail in every calculation.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

4. Explain Describe the relationship Among Fault Models.. **[3 marks]**

Fault models are related by abstraction and by the defects they can represent. A physical defect may be represented by one or more logical faults, and a single logical model may cover several physical causes. For example, a short or transistor defect can sometimes produce a stuck-at behaviour under a particular circuit condition, while a delay defect may escape a purely static stuck-at test. The relationship is therefore one of useful coverage and equivalence under assumptions, not a claim that all models are identical.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

module-2 — Logic and Fault Simulation and Testability Measures

5. Explain Explain Simulation for Design Verification and Test Evaluation. [3 marks]

Simulation applies input vectors to a circuit model and computes the resulting internal and output values. In design verification, the objective is to check whether the implementation follows the specification. In test evaluation, the objective is to determine whether a test pattern detects a targeted fault. A good simulation flow defines inputs and clocking, establishes expected responses, runs the fault-free model, introduces faults when required, and records mismatches, coverage, and unresolved cases.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

6. Explain Explain Modeling Circuits for Simulation. [3 marks]

A simulation model represents gates, interconnections, storage elements, delays, and external signals at a selected abstraction level. A combinational model maps current inputs to outputs; a sequential model also represents state, clock events, reset, and feedback. A useful test model has clear signal names, deterministic initial conditions, correct logic polarity, and a testbench that applies meaningful patterns. Modelling resolution matters because a gate-level model, switch-level model, and behavioural model expose different details and computational costs.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

7. Explain Explain the algorithms for true value and fault simulation. [3 marks]

True-value simulation evaluates the fault-free circuit for each input vector. Fault simulation repeats or augments that computation for circuits containing targeted faults. A fault is activated when the good value at its site differs from the faulty value; it is propagated when a sensitised path carries that difference to a primary output or scan observation point. Fault lists can be reduced using equivalence or dominance assumptions, and detected faults can be dropped from later simulation. The Boolean Difference Method relates output sensitivity to an input or internal variable; path sensitization selects non-controlling values on side inputs so the fault effect can pass through a gate; the Kohavi Algorithm is an algorithmic procedure for logic-fault analysis and test generation.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

8. Explain Find the faults in the given logic circuit. [3 marks]

Finding faults in a logic circuit follows a repeatable sequence. Identify the fault site and fault type, determine the good value required to activate it, choose values on side inputs that avoid masking, propagate the difference through a path, and observe the output. If no path can be sensitised, the fault may be redundant for the chosen observation or equivalent to another fault. For sequential logic, the procedure also includes state initialisation, clock sequencing, and observation over time. A fault table records the vector, expected good output, faulty output, and detection status.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

module-3 — Test Generation for Combinational and Sequential Circuits

9. Explain Explain Combinational ATPG Algorithms. [3 marks]

Combinational ATPG finds an input vector that activates a fault and propagates its effect to a primary output. Common reasoning steps are fault activation, forward implication, backward implication, path sensitization, and conflict resolution. Algorithms may use circuit topology, logic values, D-algebra or related symbolic representations, and systematic backtracking. A valid result must satisfy both the activation condition and the observation condition; if every possible path is blocked, the fault is redundant or untestable under the model.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

10. Explain Explain Sequential ATPG Algorithms. [3 marks]

Sequential ATPG must control and observe circuit state as well as primary inputs. A test sequence may require an initialisation or reset operation, several capture cycles, and a final observation cycle. The algorithm searches through state transitions, uses time-frame expansion or state justification, and accounts for feedback and storage elements. Scan design simplifies the problem by making internal state directly loadable and observable; without scan, the required sequence may be longer and the search space much larger.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

11. Explain Explain Simulation based ATPG. **[3 marks]**

Simulation-based ATPG uses circuit simulation to evaluate candidate test patterns and guide the search toward patterns that detect remaining faults. It may begin with random or deterministic patterns, simulate the fault list, drop detected faults, and generate additional patterns for faults that remain undetected. The method is simple to integrate with existing simulators and can quickly detect easy faults, but it may require many patterns for hard-to-detect faults and does not guarantee a minimum test set.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

12. Explain Explain Genetic Algorithm Based ATPG. **[3 marks]**

A Genetic Algorithm (GA) represents a candidate test pattern or sequence as a chromosome. A population of candidates is evaluated by a fitness function such as the number of faults detected, fault-effect propagation, or coverage gain. Selection retains stronger candidates; crossover combines portions of parent patterns; mutation introduces variation. The process repeats until a coverage target, iteration limit, or acceptable fitness is reached. GA-based ATPG can explore a large search space, but it is heuristic and needs careful encoding, fitness design, and reproducible stopping criteria.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

module-4 — Design for Testability, BIST and Fault Diagnosis

13. Explain Explain Testabilty basics and its analysis. **[3 marks]**

Testability describes how easily a circuit can be controlled from available inputs and how easily internal effects can be observed at outputs. Controllability measures the effort needed to set a node to a required logic value; observability measures the effort needed to propagate a node effect to an observation point. Testability analysis identifies hard-to-control, hard-to-observe, reconvergent, redundant, or highly sequential regions. Scan cell designs replace or augment storage elements so internal state can be shifted in and out, and scan architecture connects these cells into one or more scan chains.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

14. Explain Explain BIST architecture. **[3 marks]**

Built-In Self-Test (BIST) places test-pattern generation, response compaction or analysis, control, and pass/fail decision logic on or near the chip. In Random Logic BIST, a pseudo-random pattern generator stimulates logic and a signature register compacts responses; an on-chip controller sequences reset, pattern application, capture, and completion. BIST reduces dependence on external tester pins and can support field or power-on test, but it consumes area, power, design effort, and may suffer from aliasing or random-pattern-resistant faults.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

15. Explain Explain Fault Models for Diagnosis. **[3 marks]**

Fault diagnosis goes beyond detection: it uses failing responses to identify or rank possible fault locations and types. Diagnostic fault models may include stuck-at, bridging, transition, open, or scan-chain faults according to the observed failure mechanism. Diagnosis begins with basic definitions and a candidate fault list, generates distinguishing vectors, applies or simulates those vectors, and compares response signatures. Combinational logic diagnosis uses output differences and structural reasoning; Scan Chain Diagnosis examines shift and capture behaviour; Logic BIST diagnosis uses signatures, checkpoints, or additional diagnostic patterns to narrow the suspect region.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

Practice Model Question Paper

Practice Model Paper — Not an Official Examination Paper

This paper is generated from the supplied module coverage for revision and does not claim to reproduce the official examination pattern.

1. **1.** Define or explain *Explain VLSI Testing Process* with one relevant application. (5 marks)
2. **2.** Define or explain *Explain VLSI Testing Equipment* with one relevant application. (5 marks)
3. **3.** Define or explain *Explain Simulation for Design Verification and Test Evaluation* with one relevant application. (5 marks)
4. **4.** Define or explain *Explain Modeling Circuits for Simulation* with one relevant application. (5 marks)
5. **5.** Define or explain *Explain Combinational ATPG Algorithms* with one relevant application. (5 marks)
6. **6.** Define or explain *Explain Sequential ATPG Algorithms* with one relevant application. (5 marks)
7. **7.** Define or explain *Explain Testabilty basics and its analysis* with one relevant application. (5 marks)
8. **8.** Define or explain *Explain BIST architecture* with one relevant application. (5 marks)

> **Writing advice:** Use headings, standard terminology, and labelled diagrams or procedures where relevant.

Quick Revision

Module-wise key points

- **Introduction to VLSI Testing and Fault Models:** Module 1 establishes the testing vocabulary: process, equipment, quality, abstraction, and the limits of each fault model.
- **Logic and Fault Simulation and Testability Measures:** Module 2 turns fault models into observable evidence through good-value simulation, faulty-value simulation, activation, propagation, and testability reasoning.
- **Test Generation for Combinational and Sequential Circuits:** Module 3 compares systematic and heuristic test-generation approaches for memoryless and stateful digital circuits.
- **Design for Testability, BIST and Fault Diagnosis:** Module 4 makes internal circuitry testable and uses structured response information to detect, localise, and diagnose faults.

Exam checklist

- Write the official terminology before adding explanation.
- Use a labelled diagram or step sequence when it improves the answer.

- Check conditions, boundaries, safety, hygiene, and documentation points.
- Separate official syllabus information from your own example.

Last-minute revision: Review the coverage table, formula bank, diagram library, and common-mistake notes inside every topic.

Glossary

Technical glossary

Term	Short meaning
Explain VLSI Testing Process	The VLSI testing process converts a design specification into test requirements, derives test patterns, applies them to a device, captures the responses, compares the responses with expected values, and records pass/fail information. Testing may occur at wafer
Explain VLSI Testing Equipment	Automatic Test Equipment (ATE) supplies controlled input patterns, timing, power, and measurement functions to a device under test. A typical setup includes a tester controller, pattern memory, timing and signal-generation resources, power supplies, pin electr
Explain fault models	A fault model is a simplified logical representation of a physical defect. Common digital models include stuck-at-0 and stuck-at-1 faults, open faults, short or bridging faults, transistor-level defects, and delay faults. The stuck-at model assumes that a sign
Describe the relationship Among Fault Models.	Fault models are related by abstraction and by the defects they can represent. A physical defect may be represented by one or more logical faults, and a single logical model may cover several physical causes. For example, a short or transistor defect can somet
Explain Simulation for Design Verification and Test Evaluation	Simulation applies input vectors to a circuit model and computes the resulting internal and output values. In design verification, the objective is to check whether the implementation follows the specification. In test evaluation, the objective is to determine
Explain Modeling Circuits for Simulation	A simulation model represents gates, interconnections, storage elements, delays, and external signals at a selected abstraction level. A combinational model maps current inputs to outputs; a sequential model also represents state, clock events, reset, and feed
Explain the algorithms for true value and fault simulation	True-value simulation evaluates the fault-free circuit for each input vector. Fault simulation repeats or augments that computation for circuits containing targeted faults. A fault is activated when the good value at its site differs from the faulty value; it

Term	Short meaning
Find the faults in the given logic circuit	Finding faults in a logic circuit follows a repeatable sequence. Identify the fault site and fault type, determine the good value required to activate it, choose values on side inputs that avoid masking, propagate the difference through a path, and observe the
Explain Combinational ATPG Algorithms	Combinational ATPG finds an input vector that activates a fault and propagates its effect to a primary output. Common reasoning steps are fault activation, forward implication, backward implication, path sensitization, and conflict resolution. Algorithms may u
Explain Sequential ATPG Algorithms	Sequential ATPG must control and observe circuit state as well as primary inputs. A test sequence may require an initialisation or reset operation, several capture cycles, and a final observation cycle. The algorithm searches through state transitions, uses ti
Explain Simulation based ATPG	Simulation-based ATPG uses circuit simulation to evaluate candidate test patterns and guide the search toward patterns that detect remaining faults. It may begin with random or deterministic patterns, simulate the fault list, drop detected faults, and generate
Explain Genetic Algorithm Based ATPG	A Genetic Algorithm (GA) represents a candidate test pattern or sequence as a chromosome. A population of candidates is evaluated by a fitness function such as the number of faults detected, fault-effect propagation, or coverage gain. Selection retains stronge
Explain Testabilty basics and its analysis	Testability describes how easily a circuit can be controlled from available inputs and how easily internal effects can be observed at outputs. Controllability measures the effort needed to set a node to a required logic value; observability measures the effort
Explain BIST architecture	Built-In Self-Test (BIST) places test-pattern generation, response compaction or analysis, control, and pass/fail decision logic on or near the chip. In Random Logic BIST, a pseudo-random pattern generator stimulates logic and a signature register compacts res
Explain Fault Models for Diagnosis	Fault diagnosis goes beyond detection: it uses failing responses to identify or rank possible fault locations and types. Diagnostic fault models may include stuck-at, bridging, transition, open, or scan-chain faults according to the observed failure mechanism.

Official References and Resources

References listed in the supplied syllabus

1. T1 — Laung-Terng Wang, Cheng-Wen Wu and Xiaoqing Wen, “VLSI Test Principles and Architectures”, Elsevier, 2017.
2. T2 — Michael L. Bushnell and Vishwani D. Agrawal, “Essentials of Electronic Testing for Digital, Memory & Mixed-Signal VLSI Circuits”, Kluwer Academic Publishers, 2017.

3. R1 — Niraj K. Jha and Sandeep Gupta, “Testing of Digital Systems”, Cambridge University Press, 2017.
4. R2 — Parag K. Lala, “An Introduction to Logic Circuit Testing,” Morgan Claypool Publishers, 2009.
5. R3 — Parag K. Lala, “Digital Circuit Testing and Testability,” Academic Press, 1997.

Official learning resources listed

- https://onlinecourses.nptel.ac.in/noc21_ee39
- https://onlinecourses.nptel.ac.in/noc25_ee24/preview
-

In-page Search
